E. Generative Modeling

E.1 Generative Modeling: Basic concepts

E.2 Generative Modeling: Variational inference, VAE

Slightly math heavy lecture

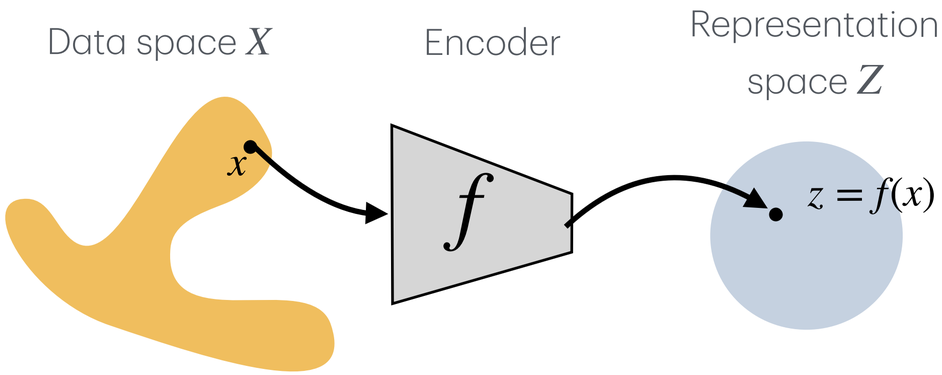
E.3 Generative Modeling: Diffusion models

E.4 Generative Modeling: Flow models

Tiam Heydari, PhD
Postdoctoral Fellow, UBC

What is generative modeling?

Representation Learning



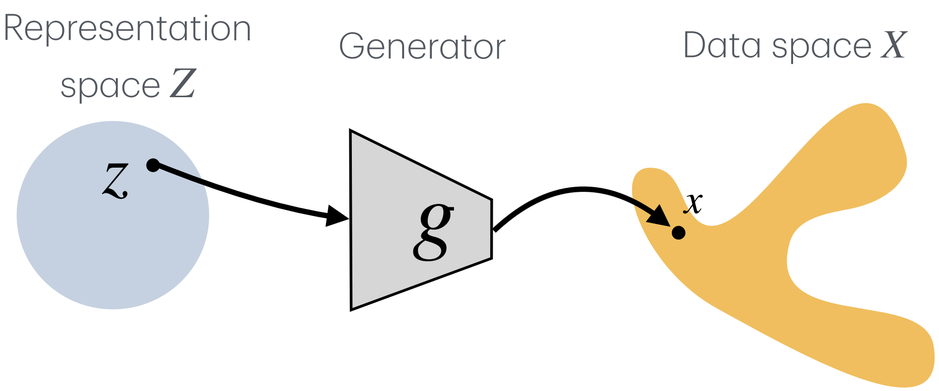
$$f : \mathcal{X} \to Z$$
$$z = f(x)$$

Question: Given data, what representation describes it?

Main direction: data → latent representation

Goal: compress, organize, compare, or predict from data.

Generative Modeling



$$g : Z \to \mathcal{X}$$
$$x = g(z)$$

Question: Given a latent code, what data could it produce?

Main direction: latent representation → data

Goal: generate realistic new samples based on some variables

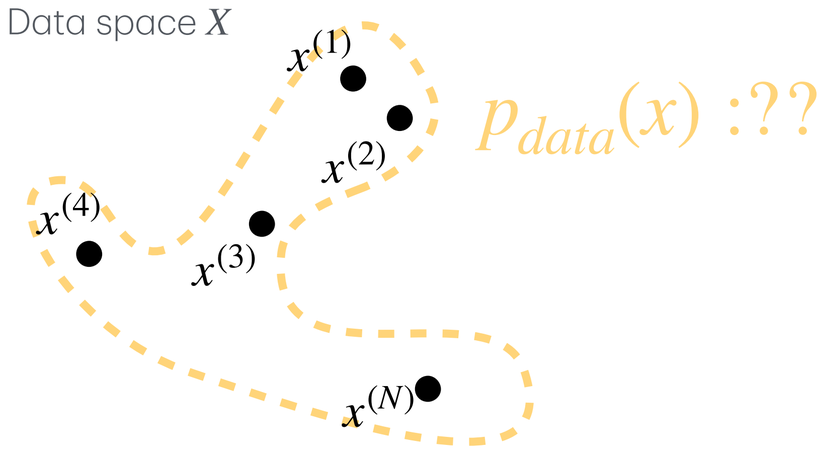
What problem we are trying to solve?

At the start of generative modeling, all we have is a finite collection of data points:

$$x^{(1)}, x^{(2)}, \ldots, x^{(N)} \in \mathcal{X}$$

we assume they were sampled from some unknown distribution $p_{data}(x)$,

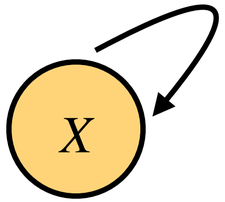
The challenge is we assume the data is coming from $p_{data}(x)$, but we don't have access to $p_{data}(x)$



During this lecture, we will see some mathematical gymnastics and some computational tricks to solve this problem (finding proper $p_{data}(x)$) approximately by a model.

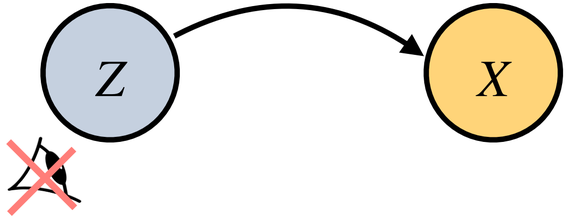
Step 1: Convert the problem into a latent space problem

We can't write down $P_{data}(x)$ directly, it's too complex. So we **invent** a hidden variable z with a simple prior distributions $P(z)$, and let a generator turn z to x.



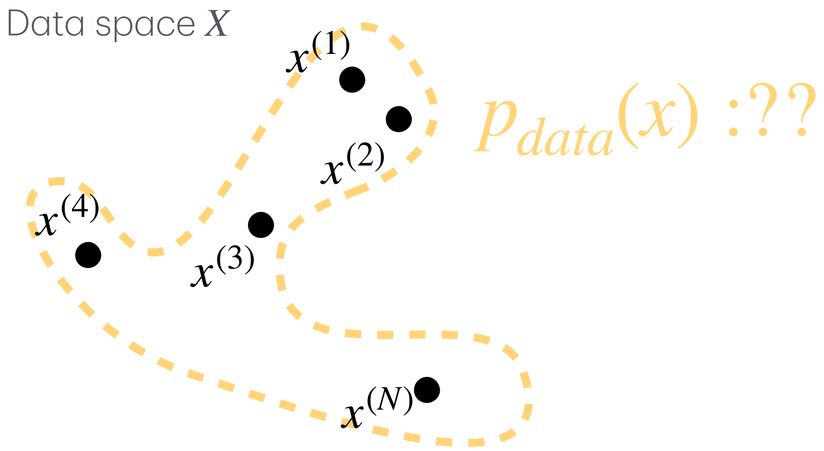
X is observable

$P_{data}(x)$ is complex
or b learned directly

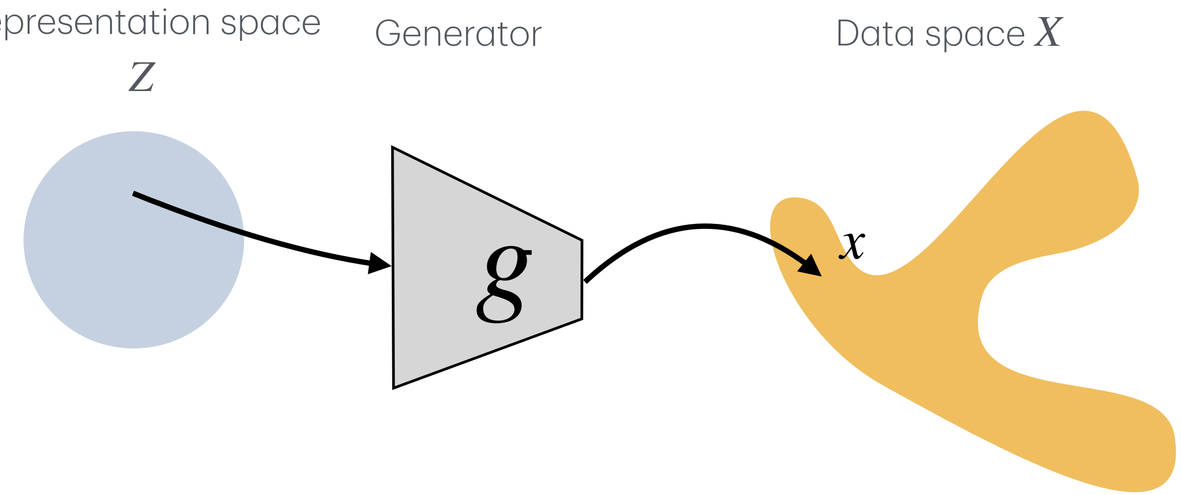


Z is not observable

$P(z)$ called **prior**. could be simple, We choose a simple Gaussian



$p_{data}(x)$:??



Step 2: The generator is an infinite mixture of gaussians

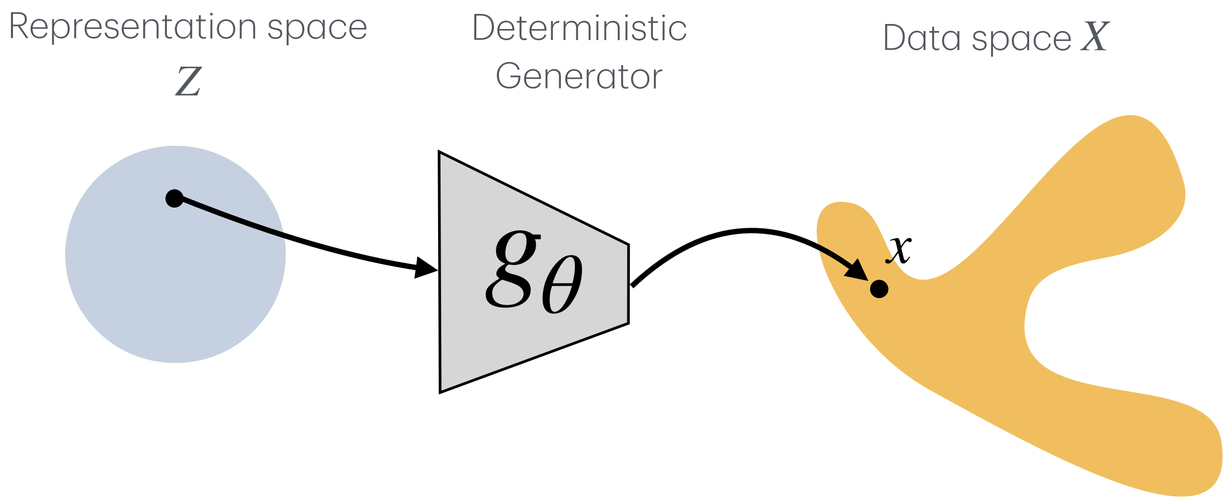
A deterministic generator is too thin

If the generator is only a deterministic function

$$x = g_\theta(z),$$

then each latent point z produces exactly one output x. So the model distribution is just the prior $p(z)$ pushed through g_θ:

$$z \sim p(z), \qquad x = g_\theta(z)\,.$$



This can generate samples, but it gives no local uncertainty around each generated point. Remember the physics case study of previous lecture.

Step 2: The generator is an infinite mixture of gaussians

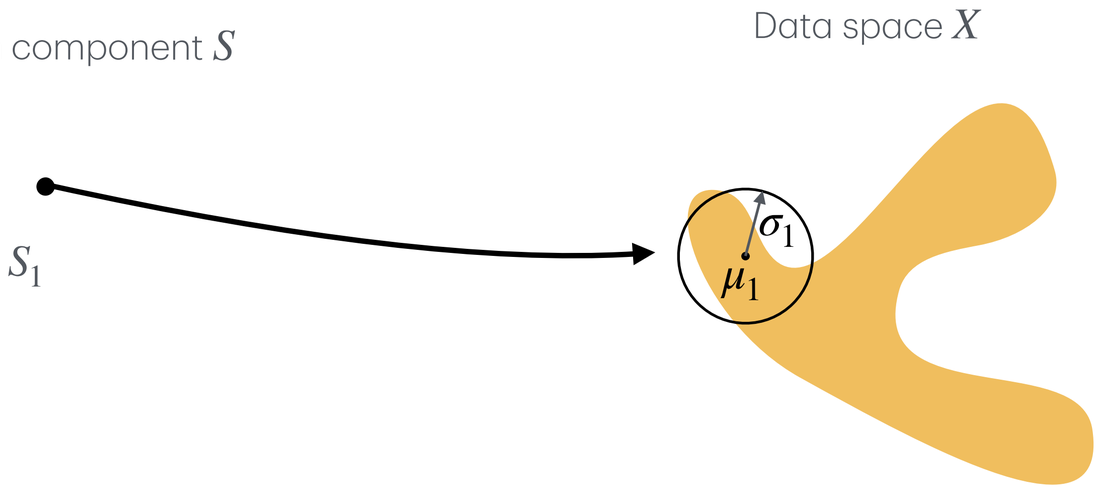
A finite mixture model adds uncertainty, but is too rigid

A simple way to model complex data is to use a mixture of K Gaussian components.

First choose a hidden component: $s \in \{1,2,\ldots,K\}$.

So the full data distribution is the average of all K Gaussian blobs: $p(x) = \frac{1}{K}\sum_{k=1}^{K} \mathcal{N}(x;\mu_k,\sigma_k)$

This is better than a deterministic generator.



Step 2: The generator is an infinite mixture of gaussians

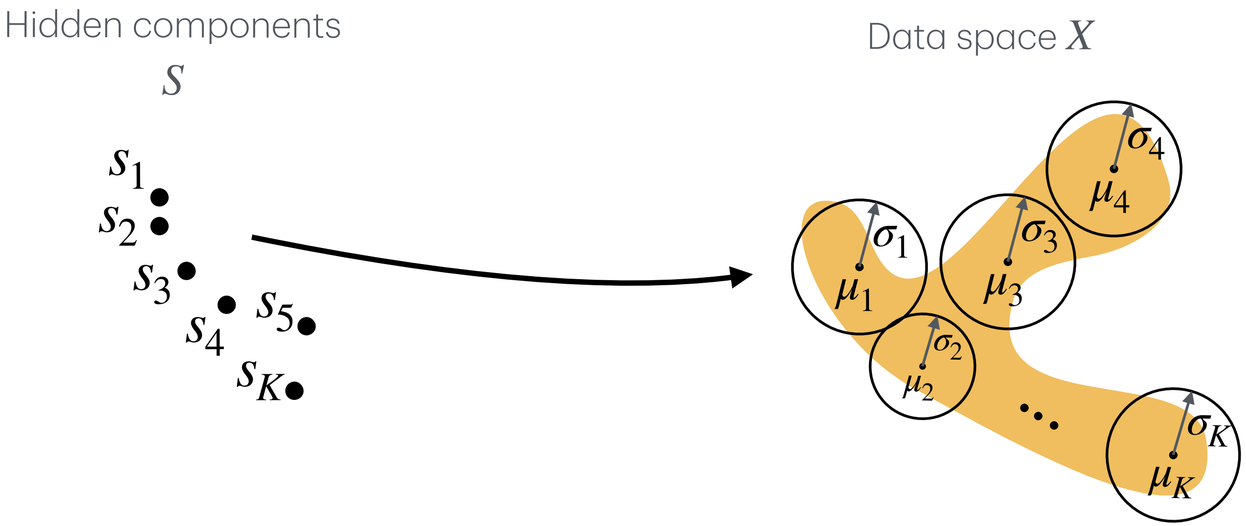
A finite mixture model adds uncertainty, but is too rigid

A simple way to model complex data is to use a mixture of K Gaussian components.

First choose a hidden component: $s \in \{1,2,\ldots,K\}$.

So the full data distribution is the average of all K Gaussian blobs: $p(x) = \frac{1}{K}\sum_{k=1}^{K} \mathcal{N}(x; \mu_k, \sigma_k)$

This is better than a deterministic generator.



But it is still limited. Each blob has its own free mean and variances (μ_k, σ_k), and they are seemingly disconnected from each other.

Step 2: The generator is an infinite mixture of gaussians

An infinite mixture of gaussian model adds uncertainty, and is flexible and smooth

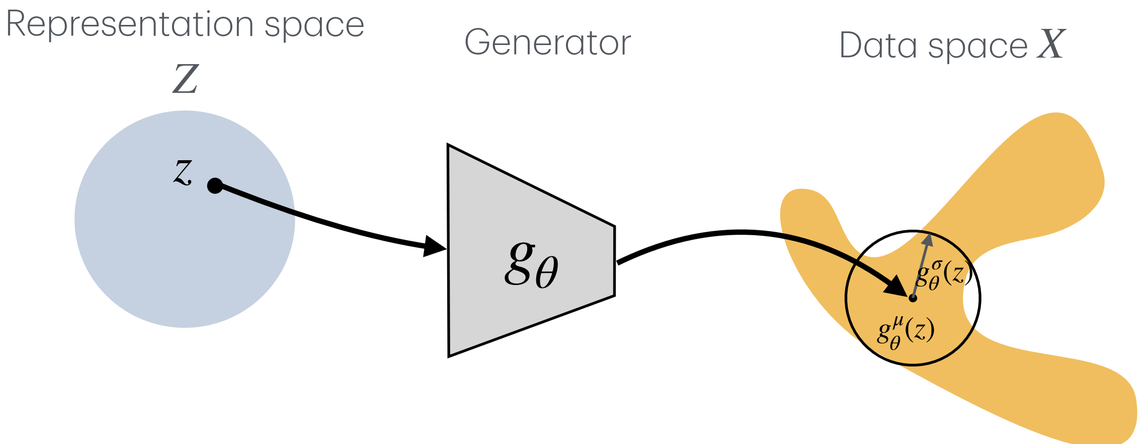
We can replace individual mean and variances (μ_k, σ_k), with an smooth and learnable function.

This function, acts on the continuous latents domain Z, and for each point of it $z \in Z$ generates the parameters we need to define a gaussian as a function of the input point. Note that g_θ should output two things, mean and var:

$$g_\theta^\mu(z) \text{ and } g_\theta^\sigma(z) \text{ for } z \in Z$$

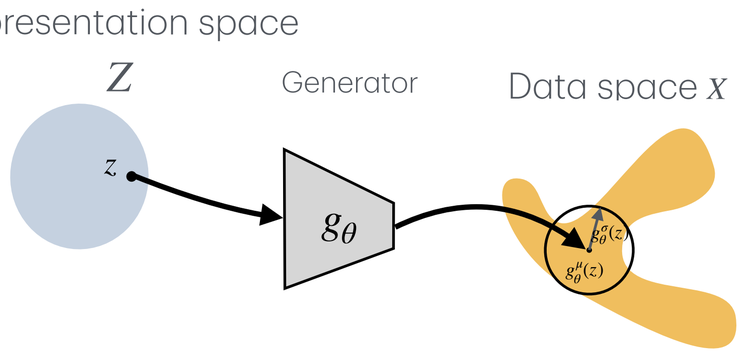
Trick#1 Means a deterministic function g_θ parametrize a stochastic gaussian function! This is called Reparameterization trick.

- we sample x from a Gaussian: $p_\theta(x \mid z) = \mathcal{N}\left(x; g_\theta^\mu(z), \left(g_\theta^\sigma(z)\right)\right)$
- Equivalently, we can write the sampling process as: $\epsilon \sim \mathcal{N}(0,I),\ x = g_\theta^\mu(z) + g_\theta^\sigma(z)\epsilon$



Step 2: The generator is an infinite mixture of gaussians

How do we train this model (g_θ) based on our limited observations, x_i?



Remember the maximum likelihood: $\theta^* = \max_\theta \sum_{i=1}^{N} \log p_\theta(x_i)$

Now remember marginalization: $p_\theta(x_i) = \int p_\theta(x_i \mid z)p(z)\,dz$

Therefore the maximum likelihood becomes: $\theta^* = \max_\theta \sum_{i=1}^{N} \log \int p_\theta(x_i \mid z)p(z)\,dz$

Using the Gaussian decoder: $\theta^* = \max_\theta \sum_{i=1}^{N} \log \int \mathcal{N}\left(x_i; g_\theta^\mu(z), \left(g_\theta^\sigma(z)\right)\right) p(z)\,dz$

What does this mean? For each data point x_i, we do not know which latent z produced it. So we will checks all possible zs in $p_\theta(x_i) = \int p_\theta(x_i \mid z)p(z)\,dz$.

Problem#1: The model is conceptually simple: average over all possible latent explanations. The problem is that doing the integral over infinite values of z exactly, is too expensive and impossible to compute.

Step 3: Montecarlo Sampling instead of the intractable integral

Trick#2: Monte Carlo instead of the integral

To be able to calculate : $\theta^* = \max_\theta \sum_{i=1}^{N} \log p_\theta(x_i) = \max_\theta \sum_{i=1}^{N} \log \int p_\theta(x_i \mid z) p(z)\, dz$

We had to calcite this integration: $p_\theta(x_i) = \int p_\theta(x_i \mid z) p(z)\, dz$

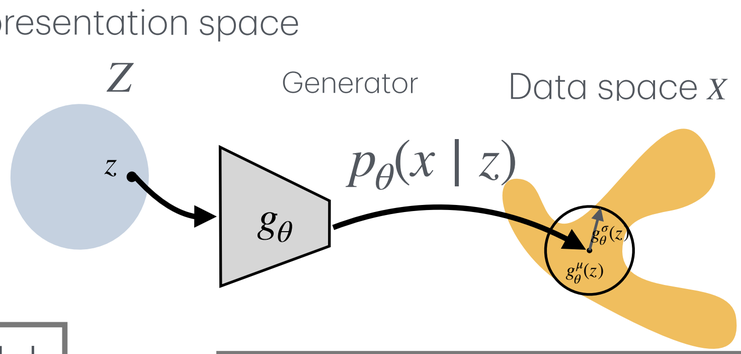
We can estimate this integral by sampling latent points from the prior: $z^{(m)} \sim p(z)$

Then: $p_\theta(x_i) \approx \frac{1}{M} \sum_{m=1}^{M} p_\theta(x_i \mid z^{(m)})$

So the likelihood objective becomes approximately:

$$\sum_{i=1}^{N} \log p_\theta(x_i) \approx \sum_{i=1}^{N} \log \left[\frac{1}{M} \sum_{m=1}^{M} p_\theta(x_i \mid z^{(m)}) \right]$$

Where we are so far? We have a full generative model



We have defined a full generative model:

$z \sim p(z)$: $p(z)$ is a prior and we chose it

$x \sim p_\theta(x \mid z)$: the generative model

With reparametrization (**trick#1**)

$$p_\theta(x \mid z) = \mathcal{N}(x; g_\theta^\mu(z), (g_\theta^\sigma(z))^2)$$

To train it, we maximize the likelihood of the observed data: $\sum_i \log p_\theta(x_i)$

but each likelihood needs an integral over all possible latent explanations:

$$p_\theta(x_i) = \int p_\theta(x_i \mid z) p(z)\, dz$$

Problem#1: integral is not tractable

Trick#2, to address problem #1: Monte Carlo could help us. We sample M points from latent, $z^{(m)} \sim p(z)$. Then:

$$\theta^* = \max_\theta \sum_{i=1}^{N} \log p_\theta(x_i) \approx \sum_{i=1}^{N} \log \left[\frac{1}{M} \sum_{m=1}^{M} p_\theta(x_i \mid z^{(m)}) \right]$$

At this point, we already have a valid generative model. After training, we can generate new samples by:

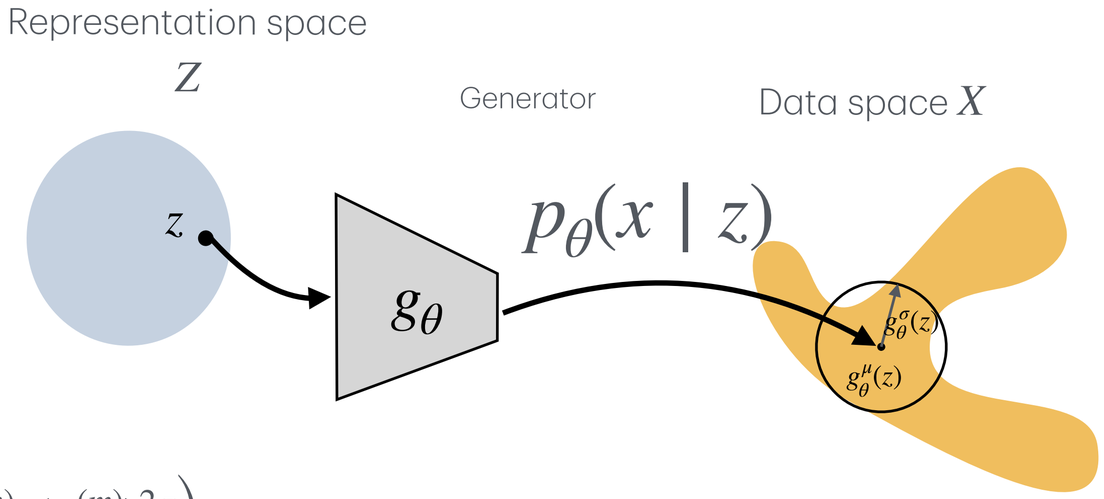
$$z \sim p(z)$$

$$x \sim p_\theta(x \mid z)$$

Where we are so far? We have a full generative model.

Example: 2D data: Arbitrary shape, here S shaped for P_{data}

2D latent: We choose a Gaussian Prior for $p(z)$



How it works? Step by step:

1) Draw latent samples from the prior: $z_{(1)}, \ldots, z_{(M)} \sim \mathcal{N}(0, I)$

2) Run the decoder on each z_m: $g_\theta(z^{(m)}) = \left(g_\theta^\mu(z^{(m)}), g_\theta^\sigma(z^{(m)}) \right)$

3) Each output defines a Gaussian blob in data space: $p_\theta(x \mid z^{(m)}) = \mathcal{N}\left(x; \mu_\theta^{(m)}, (\sigma_\theta^{(m)})^2 I \right)$

4) Score the data point (x_i) under each blob: $\ell_{i,m} = \log p_\theta(x_i \mid z^{(m)})$

5) Approximate the (Monte Carlo) likelihood of x_i for all latent samples: $p_\theta(x_i) \approx \frac{1}{M} \sum_{m=1}^{M} p_\theta(x_i \mid z^{(m)})$

6) The log-likelihood is: $\log p_\theta(x_i) \approx \log \left[\frac{1}{M} \sum_{m=1}^{M} \exp(\ell_{im}) \right]$

7) we are interested in negative log-likelihood, so grandaunt decent can optimize the θ : $\mathcal{L}(x_i) = -\log p_\theta(x_i)$

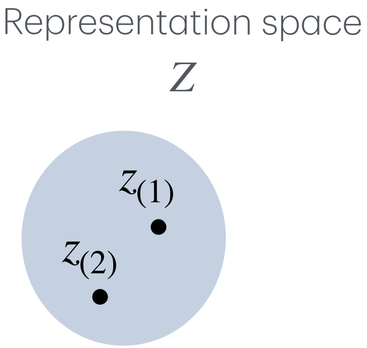
8) do this for all x_i in the training batch and sum: $\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \log p_\theta(x_i)$

9) then update θ by gradient descent to approximately find a good θ^*.

Where we are so far? We have a full generative model.

Example: 2D data: Arbitrary shape, here S shaped for P_{data}

2D latent: We choose a Gaussian Prior for $p(z)$



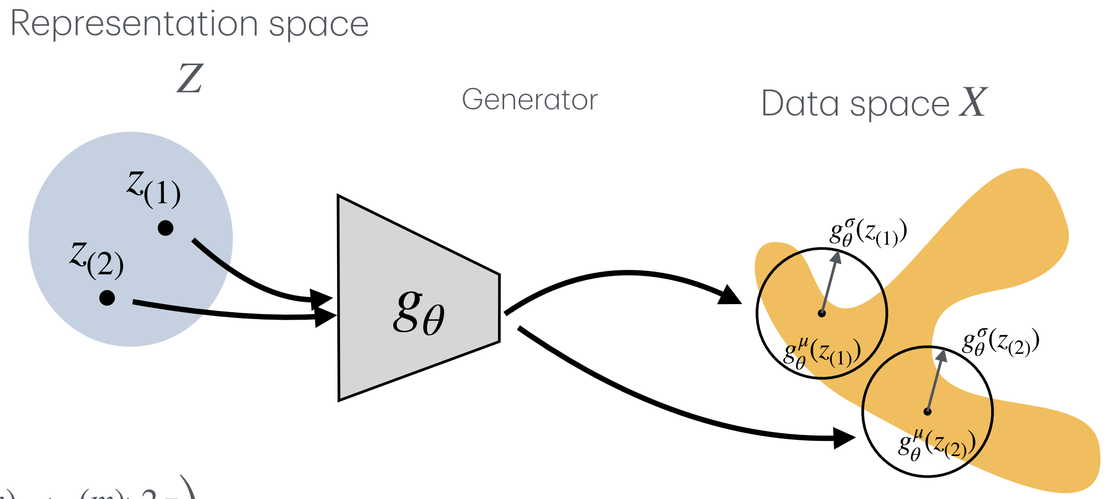
How it works? Step by step:

1) Draw latent samples from the prior: $z_{(1)}, \ldots, z_{(M)} \sim \mathcal{N}(0,I)$

Where we are so far? We have a full generative model.

Example: 2D data: Arbitrary shape, here S shaped for P_{data}

2D latent: We choose a Gaussian Prior for $p(z)$



How it works? Step by step:

1) Draw latent samples from the prior: $z_{(1)}, \ldots, z_{(M)} \sim \mathcal{N}(0, I)$

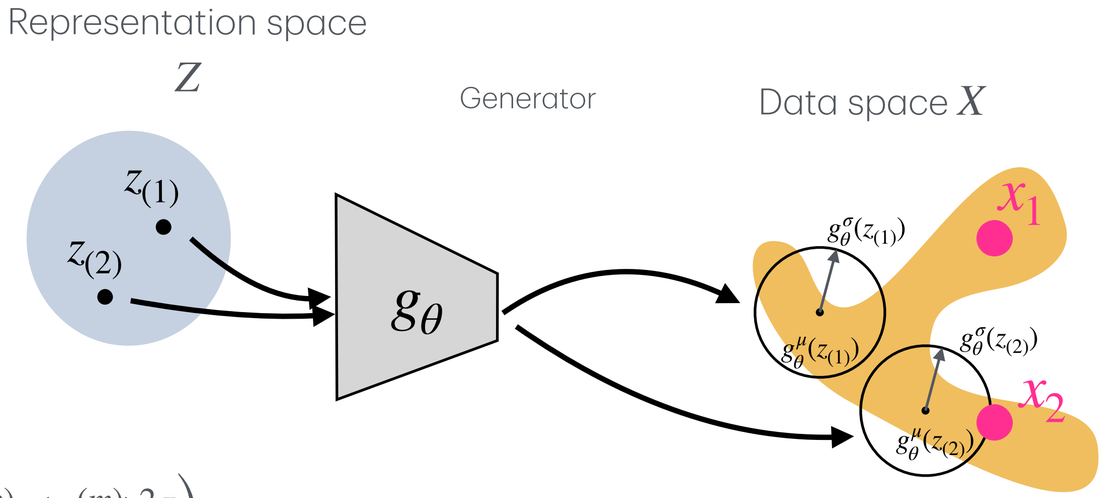
2) Run the decoder on each z_m: $g_\theta(z^{(m)}) = \left(g_\theta^\mu(z^{(m)}), g_\theta^\sigma(z^{(m)}) \right)$

3) Each output defines a Gaussian blob in data space: $p_\theta(x \mid z^{(m)}) = \mathcal{N}\left(x; \mu_\theta^{(m)}, (\sigma_\theta^{(m)})^2 I \right)$

Where we are so far? We have a full generative model.

Example: 2D data: Arbitrary shape, here S shaped for P_{data}

2D latent: We choose a Gaussian Prior for $p(z)$



How it works? Step by step:

1) Draw latent samples from the prior: $z_{(1)}, \ldots, z_{(M)} \sim \mathcal{N}(0, I)$

2) Run the decoder on each z_m: $g_\theta(z^{(m)}) = \left(g_\theta^\mu(z^{(m)}), g_\theta^\sigma(z^{(m)}) \right)$

3) Each output defines a Gaussian blob in data space: $p_\theta(x \mid z^{(m)}) = \mathcal{N}\left(x; \mu_\theta^{(m)}, (\sigma_\theta^{(m)})^2 I \right)$

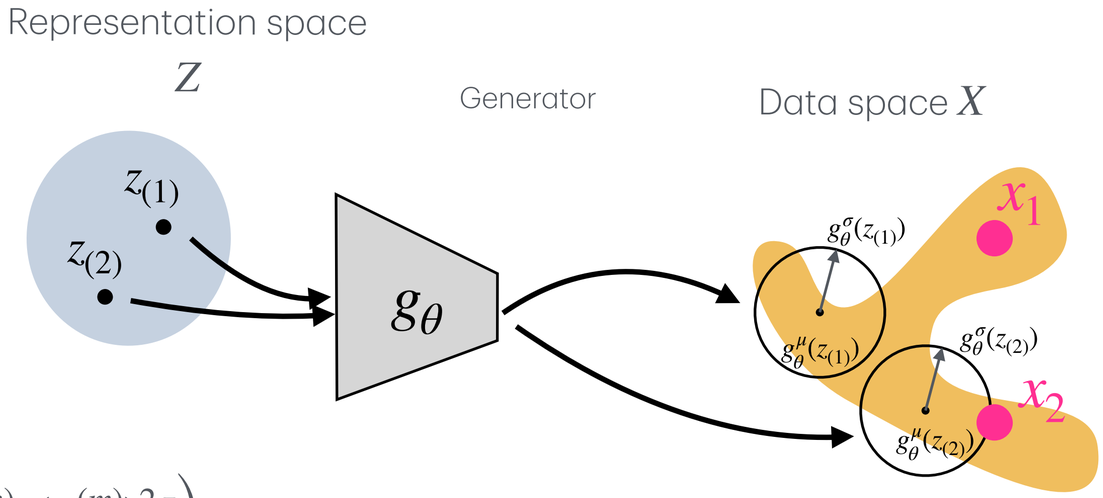
4) Score the data point (x_i) under each blob: $\ell_{i,m} = \log p_\theta(x_i \mid z^{(m)})$

$$\ell_{1,1} = \log p_\theta(x_1 \mid z_{(1)}) \approx 0$$
$$\ell_{1,2} = \log p_\theta(x_1 \mid z_{(2)}) \approx 0$$
$$\ell_{2,1} = \log p_\theta(x_2 \mid z_{(1)}) \approx 0$$
$$\ell_{2,2} = \log p_\theta(x_2 \mid z_{(2)}) > 0$$

Where we are so far? We have a full generative model.

Example: 2D data: Arbitrary shape, here S shaped for P_{data}

2D latent: We choose a Gaussian Prior for $p(z)$



How it works? Step by step:

1) Draw latent samples from the prior: $z_{(1)}, \ldots, z_{(M)} \sim \mathcal{N}(0, I)$

2) Run the decoder on each z_m: $g_\theta(z^{(m)}) = \left(g_\theta^\mu(z^{(m)}), g_\theta^\sigma(z^{(m)}) \right)$

3) Each output defines a Gaussian blob in data space: $p_\theta(x \mid z^{(m)}) = \mathcal{N}\left(x; \mu_\theta^{(m)}, (\sigma_\theta^{(m)})^2 I \right)$

4) Score the data point (x_i) under each blob: $\ell_{i,m} = \log p_\theta(x_i \mid z^{(m)})$

5) Approximate the (Monte Carlo) likelihood of x_i for all latent samples: $p_\theta(x_i) \approx \frac{1}{M} \sum_{m=1}^{M} p_\theta(x_i \mid z^{(m)})$

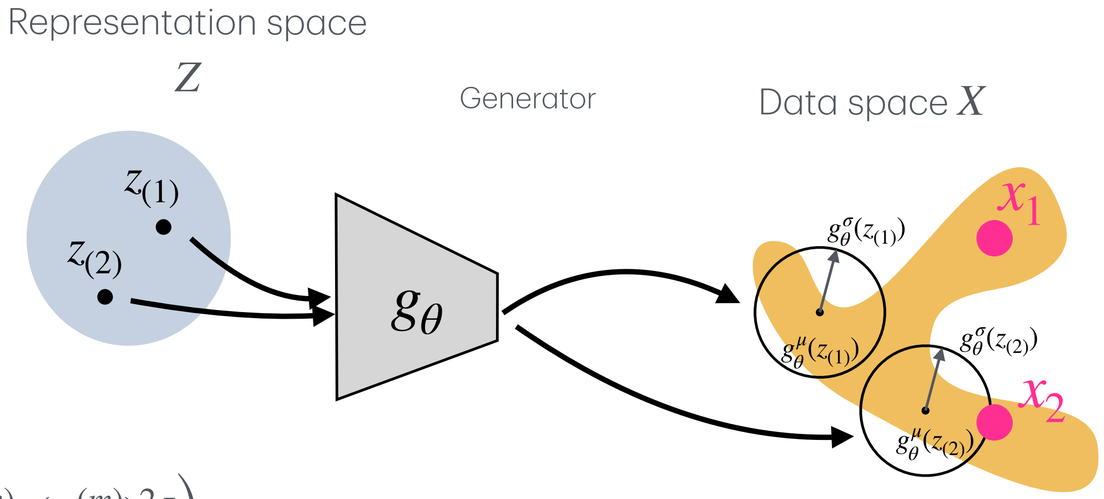
$$\left.\begin{aligned} \ell_{1,1} &= \log p_\theta(x_1 \mid z_{(1)}) \approx 0 \\ \ell_{1,2} &= \log p_\theta(x_1 \mid z_{(2)}) \approx 0 \end{aligned}\right\} \rightarrow p_\theta(x_1)$$

$$\left.\begin{aligned} \ell_{2,1} &= \log p_\theta(x_2 \mid z_{(1)}) \approx 0 \\ \ell_{2,2} &= \log p_\theta(x_2 \mid z_{(2)}) > 0 \end{aligned}\right\} \rightarrow p_\theta(x_2)$$

Where we are so far? We have a full generative model.

Example: 2D data: Arbitrary shape, here S shaped for P_{data}

2D latent: We choose a Gaussian Prior for $p(z)$



How it works? Step by step:

1) Draw latent samples from the prior: $z_{(1)}, \ldots, z_{(M)} \sim \mathcal{N}(0, I)$

2) Run the decoder on each z_m: $g_\theta(z^{(m)}) = \left(g_\theta^\mu(z^{(m)}), g_\theta^\sigma(z^{(m)})\right)$

3) Each output defines a Gaussian blob in data space: $p_\theta(x \mid z^{(m)}) = \mathcal{N}\left(x; \mu_\theta^{(m)}, (\sigma_\theta^{(m)})^2 I\right)$

4) Score the data point (x_i) under each blob: $\ell_{i,m} = \log p_\theta(x_i \mid z^{(m)})$

5) Approximate the (Monte Carlo) likelihood of x_i for all latent samples: $p_\theta(x_i) \approx \frac{1}{M} \sum_{m=1}^{M} p_\theta(x_i \mid z^{(m)})$

$$\left.\begin{aligned} \ell_{1,1} &= \log p_\theta(x_1 \mid z_{(1)}) \approx 0 \\ \ell_{1,2} &= \log p_\theta(x_1 \mid z_{(2)}) \approx 0 \end{aligned}\right\} \rightarrow p_\theta(x_1)$$

$$\left.\begin{aligned} \ell_{2,1} &= \log p_\theta(x_2 \mid z_{(1)}) \approx 0 \\ \ell_{2,2} &= \log p_\theta(x_2 \mid z_{(2)}) > 0 \end{aligned}\right\} \rightarrow p_\theta(x_2)$$

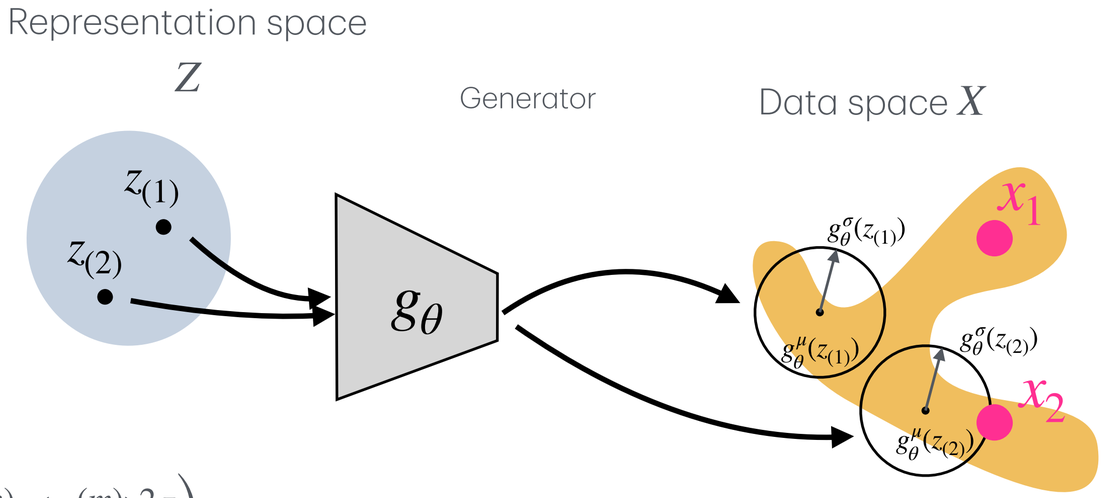
6) The log-likelihood is: $\log p_\theta(x_i) \approx \log \left[\frac{1}{M} \sum_{m=1}^{M} \exp(\ell_{im})\right]$

7) we are interested in negative log-likelihood, so grandaunt decent can optimize the θ : $\mathcal{L}(x_i) = -\log p_\theta(x_i)$

Where we are so far? We have a full generative model.

Example: 2D data: Arbitrary shape, here S shaped for P_{data}

2D latent: We choose a Gaussian Prior for $p(z)$



How it works? Step by step:

1) Draw latent samples from the prior: $z_{(1)}, \ldots, z_{(M)} \sim \mathcal{N}(0, I)$

2) Run the decoder on each z_m: $g_\theta(z^{(m)}) = \left(g_\theta^\mu(z^{(m)}), g_\theta^\sigma(z^{(m)})\right)$

3) Each output defines a Gaussian blob in data space: $p_\theta(x \mid z^{(m)}) = \mathcal{N}\left(x; \mu_\theta^{(m)}, (\sigma_\theta^{(m)})^2 I\right)$

4) Score the data point (x_i) under each blob: $\ell_{i,m} = \log p_\theta(x_i \mid z^{(m)})$

5) Approximate the (Monte Carlo) likelihood of x_i for all latent samples: $p_\theta(x_i) \approx \frac{1}{M} \sum_{m=1}^{M} p_\theta(x_i \mid z^{(m)})$

$$\left.\begin{aligned} \ell_{1,1} &= \log p_\theta(x_1 \mid z_{(1)}) \approx 0 \\ \ell_{1,2} &= \log p_\theta(x_1 \mid z_{(2)}) \approx 0 \end{aligned}\right\} \rightarrow p_\theta(x_1)$$

$$\left.\begin{aligned} \ell_{2,1} &= \log p_\theta(x_2 \mid z_{(1)}) \approx 0 \\ \ell_{2,2} &= \log p_\theta(x_2 \mid z_{(2)}) > 0 \end{aligned}\right\} \rightarrow p_\theta(x_2)$$

6) The log-likelihood is: $\log p_\theta(x_i) \approx \log \left[\frac{1}{M} \sum_{m=1}^{M} \exp(\ell_{im})\right]$

$$\left.\begin{aligned} p_\theta(x_1) &\rightarrow \mathcal{L}(x_1) \\ p_\theta(x_2) &\rightarrow \mathcal{L}(x_2) \end{aligned}\right\} \rightarrow \mathcal{L}$$

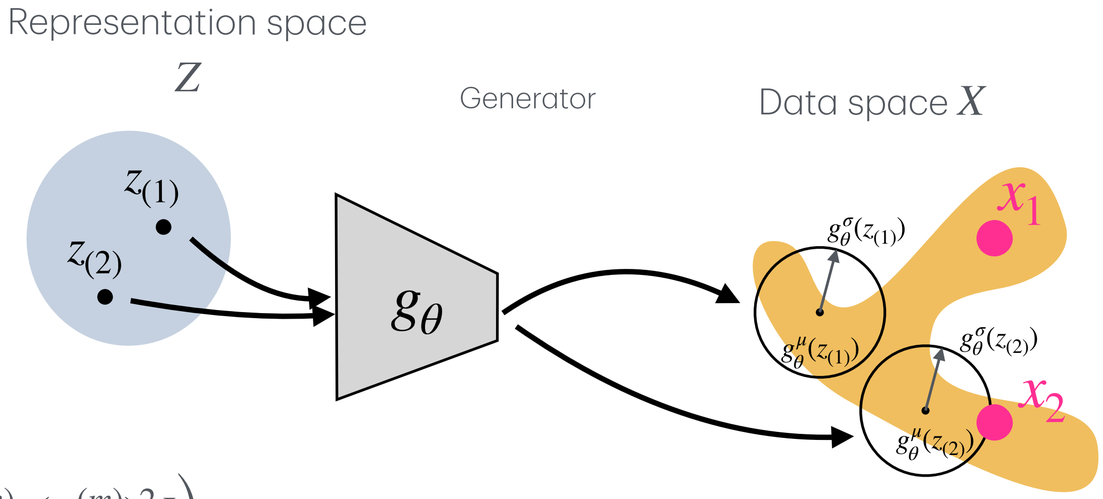
7) we are interested in negative log-likelihood, so grandaunt decent can optimize the θ : $\mathcal{L}(x_i) = -\log p_\theta(x_i)$

8) do this for all x_i in the training batch and sum: $\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \log p_\theta(x_i)$

Where we are so far? We have a full generative model.

Example: 2D data: Arbitrary shape, here S shaped for P_{data}

2D latent: We choose a Gaussian Prior for $p(z)$



How it works? Step by step:

1) Draw latent samples from the prior: $z_{(1)}, \ldots, z_{(M)} \sim \mathcal{N}(0,I)$

2) Run the decoder on each z_m: $g_\theta(z^{(m)}) = \left(g_\theta^\mu(z^{(m)}), g_\theta^\sigma(z^{(m)})\right)$

3) Each output defines a Gaussian blob in data space: $p_\theta(x \mid z^{(m)}) = \mathcal{N}\left(x; \mu_\theta^{(m)}, (\sigma_\theta^{(m)})^2 I\right)$

4) Score the data point (x_i) under each blob: $\ell_{i,m} = \log p_\theta(x_i \mid z^{(m)})$

5) Approximate the (Monte Carlo) likelihood of x_i for all latent samples: $p_\theta(x_i) \approx \frac{1}{M}\sum_{m=1}^{M} p_\theta(x_i \mid z^{(m)})$

$$\left.\begin{aligned}\ell_{1,1} &= \log p_\theta(x_1 \mid z_{(1)}) \approx 0\\ \ell_{1,2} &= \log p_\theta(x_1 \mid z_{(2)}) \approx 0\end{aligned}\right\} \rightarrow p_\theta(x_1)$$

$$\left.\begin{aligned}\ell_{2,1} &= \log p_\theta(x_2 \mid z_{(1)}) \approx 0\\ \ell_{2,2} &= \log p_\theta(x_2 \mid z_{(2)}) > 0\end{aligned}\right\} \rightarrow p_\theta(x_2)$$

6) The log-likelihood is: $\log p_\theta(x_i) \approx \log\left[\frac{1}{M}\sum_{m=1}^{M}\exp(\ell_{im})\right]$

$$\left.\begin{aligned}p_\theta(x_1) &\rightarrow \mathcal{L}(x_1)\\ p_\theta(x_2) &\rightarrow \mathcal{L}(x_2)\end{aligned}\right\} \rightarrow \mathcal{L}$$

7) we are interested in negative log-likelihood, so grandaunt decent can optimize the θ : $\mathcal{L}(x_i) = -\log p_\theta(x_i)$

8) do this for all x_i in the training batch and sum: $\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\log p_\theta(x_i)$

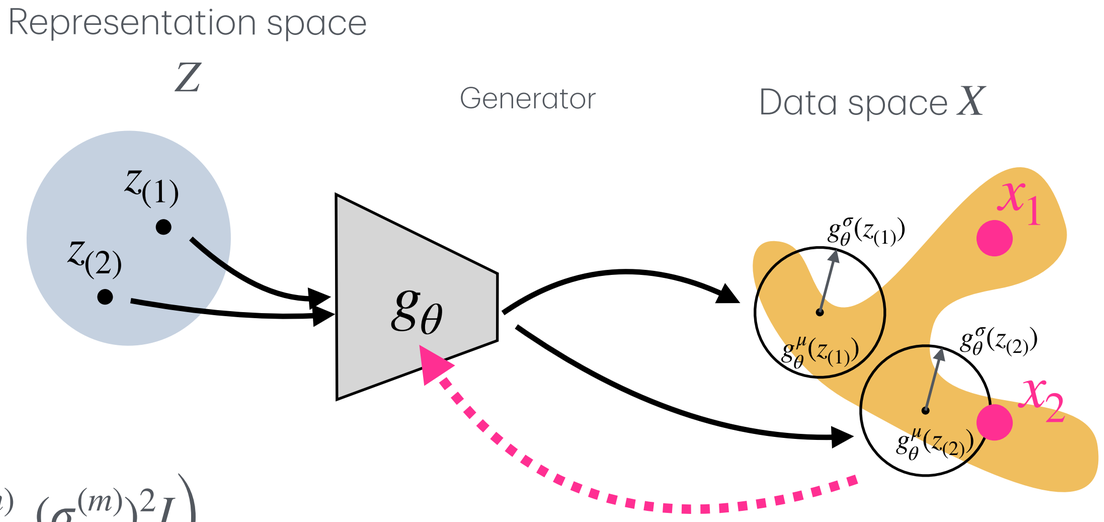
$$\mathcal{L} \rightarrow \nabla\theta \ldots \rightarrow \theta*$$

9) then update θ by gradient descent to approximately find a good $\theta*$.

Where we are so far? We have a full generative model.

Example: 2D data: Arbitrary shape, here S shaped for P_{data}

2D latent: We choose a Gaussian Prior for $p(z)$



How it works? Step by step:

1) Draw latent samples from the prior: $z_{(1)}, \ldots, z_{(M)} \sim \mathcal{N}(0, I)$

2) Run the decoder on each z_m: $g_\theta(z^{(m)}) = \left(g_\theta^\mu(z^{(m)}), g_\theta^\sigma(z^{(m)})\right)$

3) Each output defines a Gaussian blob in data space: $p_\theta(x \mid z^{(m)}) = \mathcal{N}\left(x; \mu_\theta^{(m)}, (\sigma_\theta^{(m)})^2 I\right)$

4) Score the data point (x_i) under each blob: $\ell_{i,m} = \log p_\theta(x_i \mid z^{(m)})$

5) Approximate the (Monte Carlo) likelihood of x_i for all latent samples: $p_\theta(x_i) \approx \frac{1}{M} \sum_{m=1}^{M} p_\theta(x_i \mid z^{(m)})$

$$\left.\begin{aligned} \ell_{1,1} &= \log p_\theta(x_1 \mid z_{(1)}) \approx 0 \\ \ell_{1,2} &= \log p_\theta(x_1 \mid z_{(2)}) \approx 0 \end{aligned}\right\} \rightarrow p_\theta(x_1)$$

$$\left.\begin{aligned} \ell_{2,1} &= \log p_\theta(x_2 \mid z_{(1)}) \approx 0 \\ \ell_{2,2} &= \log p_\theta(x_2 \mid z_{(2)}) > 0 \end{aligned}\right\} \rightarrow p_\theta(x_2)$$

6) The log-likelihood is: $\log p_\theta(x_i) \approx \log \left[\frac{1}{M} \sum_{m=1}^{M} \exp(\ell_{im})\right]$

$$\left.\begin{aligned} p_\theta(x_1) &\rightarrow \mathcal{L}(x_1) \\ p_\theta(x_2) &\rightarrow \mathcal{L}(x_2) \end{aligned}\right\} \rightarrow \mathcal{L}$$

7) we are interested in negative log-likelihood, so grandaunt decent can optimize the θ : $\mathcal{L}(x_i) = -\log p_\theta(x_i)$

8) do this for all x_i in the training batch and sum: $\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \log p_\theta(x_i)$

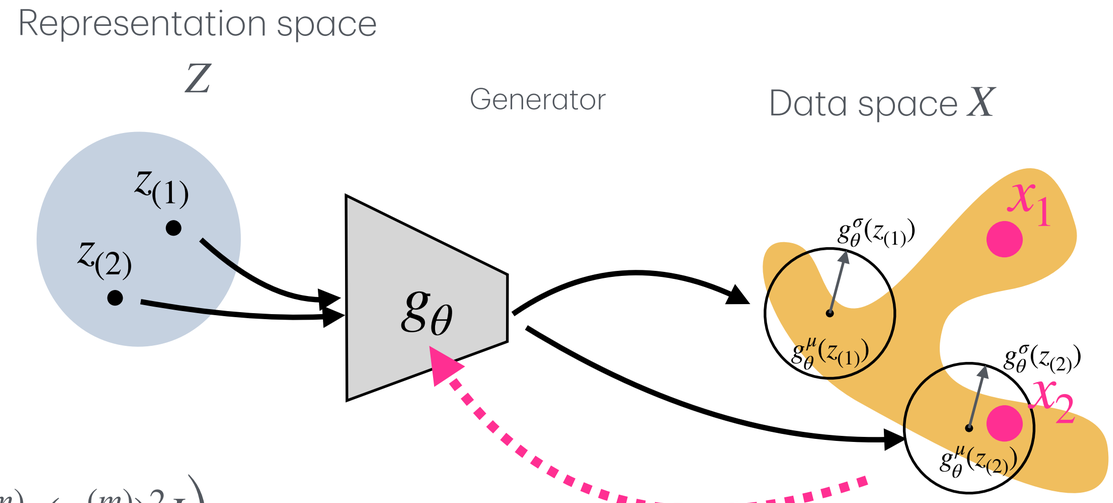
$$\mathcal{L} \rightarrow \nabla\theta \ldots \rightarrow \theta*$$

9) then update θ by gradient descent to approximately find a good $\theta*$.

Where we are so far? We have a full generative model.

Example: 2D data: Arbitrary shape, here S shaped for P_{data}

2D latent: We choose a Gaussian Prior for $p(z)$



How it works? Step by step:

1) Draw latent samples from the prior: $z_{(1)}, \ldots, z_{(M)} \sim \mathcal{N}(0, I)$

2) Run the decoder on each z_m: $g_\theta(z^{(m)}) = \left(g_\theta^\mu(z^{(m)}), g_\theta^\sigma(z^{(m)})\right)$

3) Each output defines a Gaussian blob in data space: $p_\theta(x \mid z^{(m)}) = \mathcal{N}\left(x; \mu_\theta^{(m)}, (\sigma_\theta^{(m)})^2 I\right)$

4) Score the data point (x_i) under each blob: $\ell_{i,m} = \log p_\theta(x_i \mid z^{(m)})$

5) Approximate the (Monte Carlo) likelihood of x_i for all latent samples: $p_\theta(x_i) \approx \frac{1}{M} \sum_{m=1}^{M} p_\theta(x_i \mid z^{(m)})$

$$\left.\begin{aligned} \ell_{1,1} &= \log p_\theta(x_1 \mid z_{(1)}) \approx 0 \\ \ell_{1,2} &= \log p_\theta(x_1 \mid z_{(2)}) \approx 0 \end{aligned}\right\} \rightarrow p_\theta(x_1)$$

$$\left.\begin{aligned} \ell_{2,1} &= \log p_\theta(x_2 \mid z_{(1)}) \approx 0 \\ \ell_{2,2} &= \log p_\theta(x_2 \mid z_{(2)}) > 0 \end{aligned}\right\} \rightarrow p_\theta(x_2)$$

6) The log-likelihood is: $\log p_\theta(x_i) \approx \log \left[\frac{1}{M} \sum_{m=1}^{M} \exp(\ell_{im})\right]$

$$\left.\begin{aligned} p_\theta(x_1) &\rightarrow \mathcal{L}(x_1) \\ p_\theta(x_2) &\rightarrow \mathcal{L}(x_2) \end{aligned}\right\} \rightarrow \mathcal{L}$$

7) we are interested in negative log-likelihood, so grandaunt decent can optimize the θ : $\mathcal{L}(x_i) = -\log p_\theta(x_i)$

8) do this for all x_i in the training batch and sum: $\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \log p_\theta(x_i)$

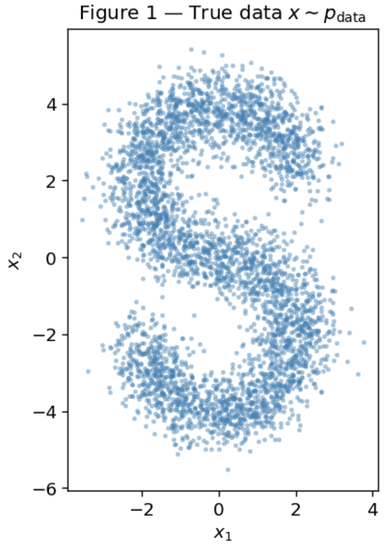
$$\mathcal{L} \rightarrow \nabla\theta \ldots \rightarrow \theta*$$

9) then update θ by gradient descent to approximately find a good $\theta*$.

Where we are so far? We have a full generative model.

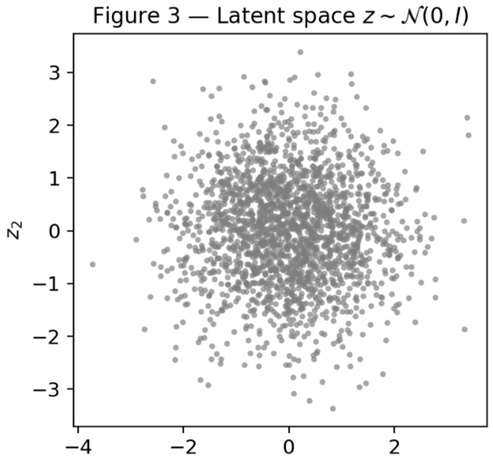
Input Data

X

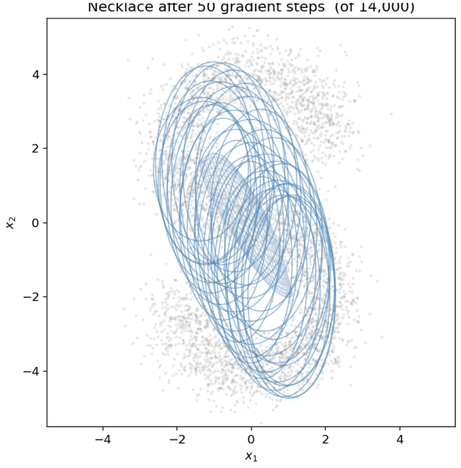


Training

Z



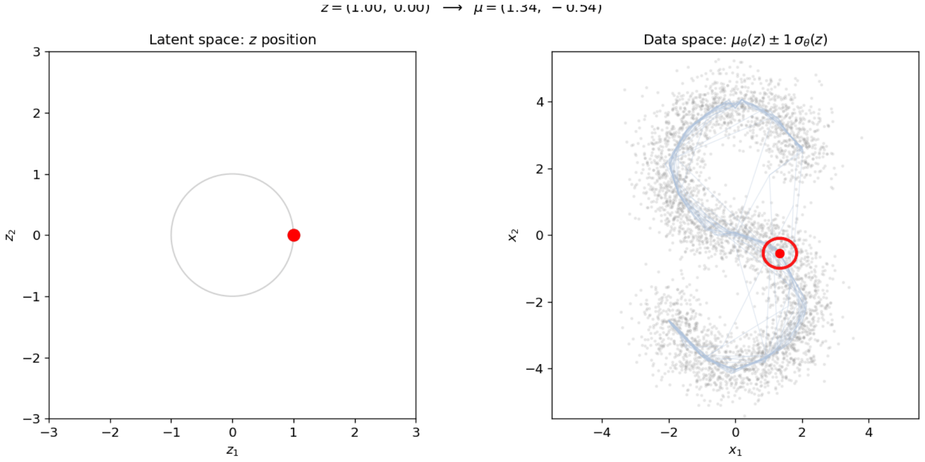
$p_\theta(x \mid z)$



Learned model generates new data

Z

$p_\theta(x \mid z)$



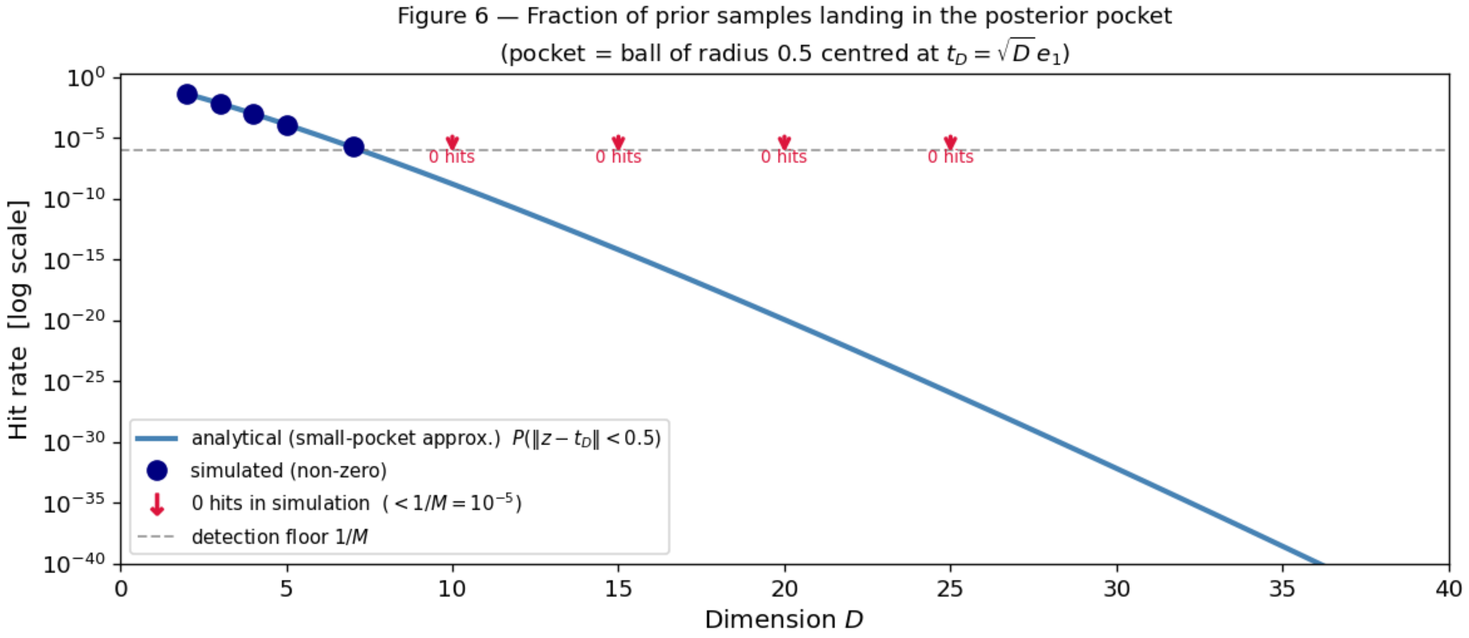
Where we are so far? We have a full generative model. …. **But is it useful in real life applications?!**

Problem#2: curse of dimensionality:

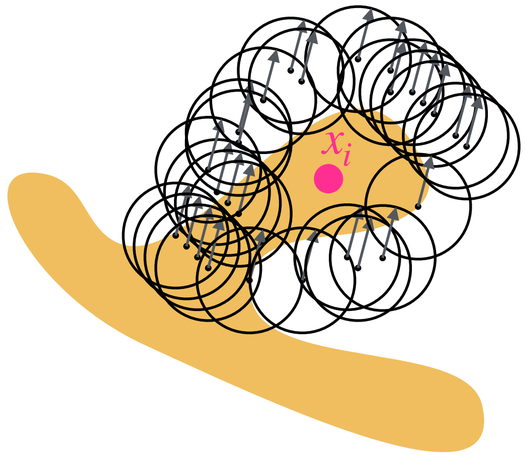
We already have a valid generative model: $z \sim p(z), \quad x \sim p_\theta(x \mid z)$

and we can train it by Monte Carlo maximum likelihood: $p_\theta(x_i) \approx \frac{1}{M} \sum_{m=1}^{M} p_\theta(x_i \mid z^{(m)}), \qquad z^{(m)} \sim p(z)$

For one data point x_i, only a small region of latent space explains it well. But Monte Carlo does not know where it is and randomly samples from simple $p(z)$. In high dimensions, almost all random prior samples ($z_{(m)}$) miss the useful region $p_\theta(x_i \mid z_{(m)}) \approx 0$, making the gradient ($\nabla\theta$) basically zero, or very noisy.

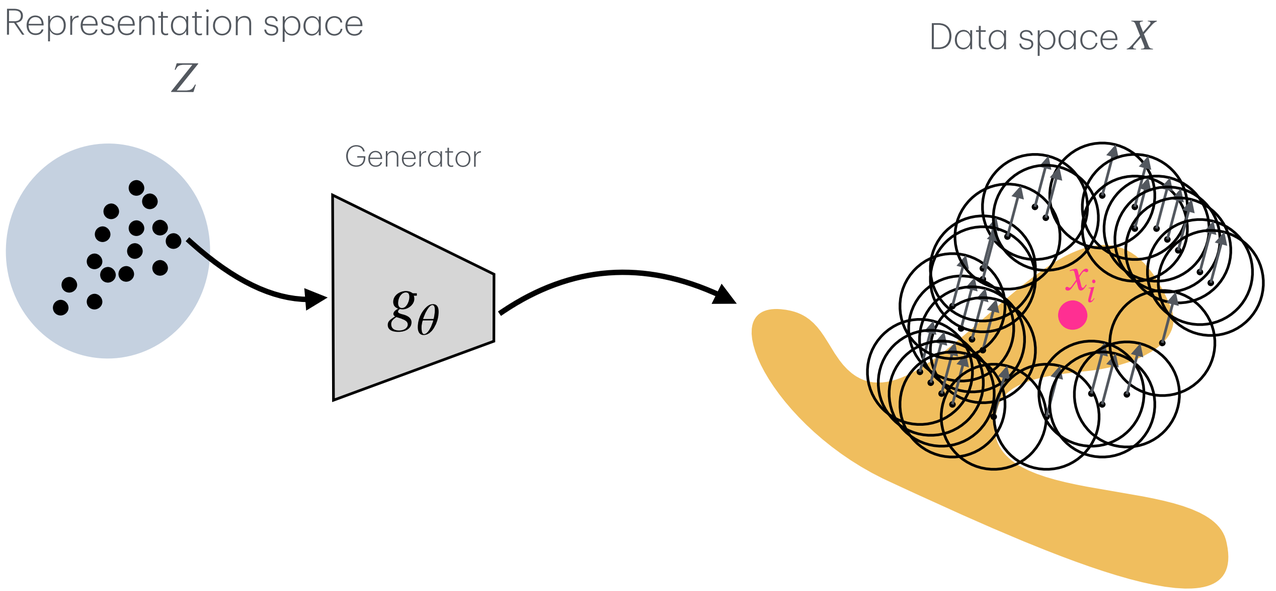


In high dimensions, most prior samples decode to regions of data space that do not explain x_i



Step 4: Importance Sampling instead of naive Montecarlo Sampling

Problem#2: (in high dimensions)

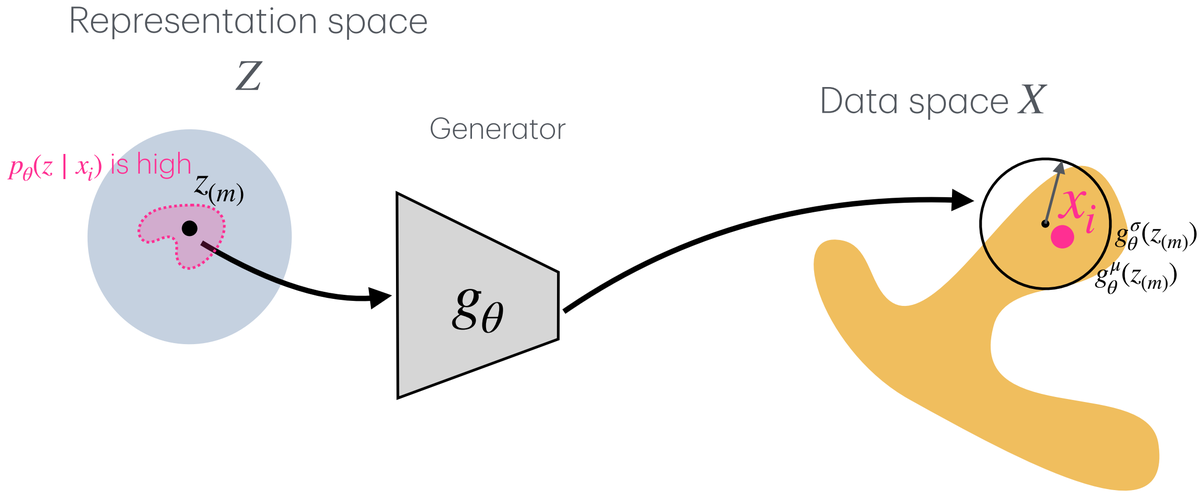


Trick#3: Sample from the useful latent region

For a given data point x_i , not all latent points z are useful to sample from. The useful region is where the posterior is high:

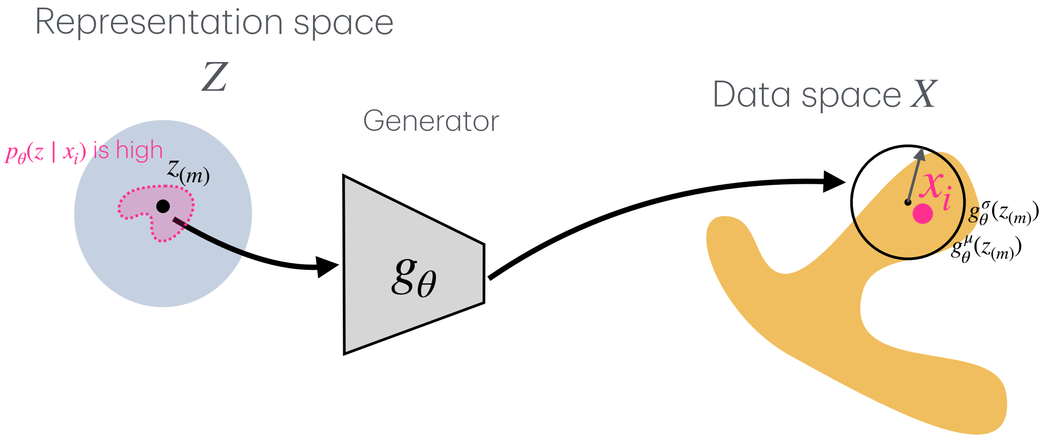
$p_\theta(z \mid x_i)$ is high

This posterior answers: which latent points z could have generated x_i?



Step 4: Importance Sampling instead of naive Montecarlo Sampling

Trick#3: The posterior is the ideal sampling distribution



Note that it is not a new function. It is derived from Bayes rule using the model we already defined (hence the parameter θ):

$$p_\theta(z \mid x_i) = \frac{p_\theta(x_i \mid z)p(z)}{p_\theta(x_i)}$$

where: $p_\theta(x_i) = \int p_\theta(x_i \mid z)p(z)\,dz$

Ideally, we would sample from: $z^{(m)} \sim p_\theta(z \mid x_i)$

Problem #3: $p_\theta(x_i) = \int p_\theta(x_i \mid z)p(z)\,dz$ is is exactly the hard integral we were trying to avoid in Problem#1, so we can't access $p_\theta(z \mid x_i)$.

Step 4: Importance Sampling instead of naive Montecarlo Sampling

Trick#4: Variational approximation

Instead of $p_\theta(z \mid x_i)$ we introduce a new learnable approximation called "encoder":

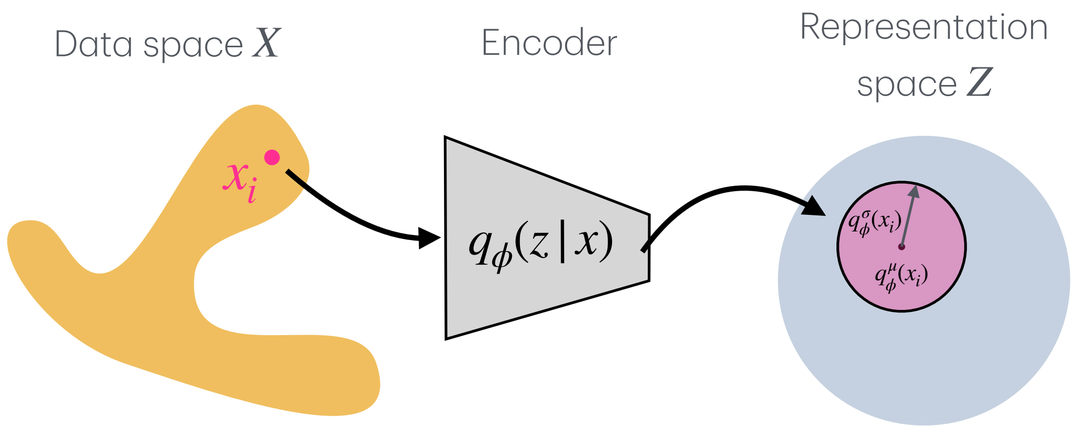
$$q_\phi(z \mid x_i) \approx p_\theta(z \mid x_i)$$

The encoder takes a data point and outputs a distribution in latent space:

$$x_i \longrightarrow q_\phi(z \mid x_i)$$

Also, for simplicity, we assume that encoder is parametrizing a gaussian probability:

$$q_\phi(z \mid x_i) = \mathcal{N}\left(z; q_\phi^\mu(x_i), \left(q_\phi^\sigma(x_i)\right)\right)$$



Step 4: Importance Sampling instead of naive Montecarlo Sampling

How do we train q_ϕ and g_θ?

We start again from maximizing the data likelihood:

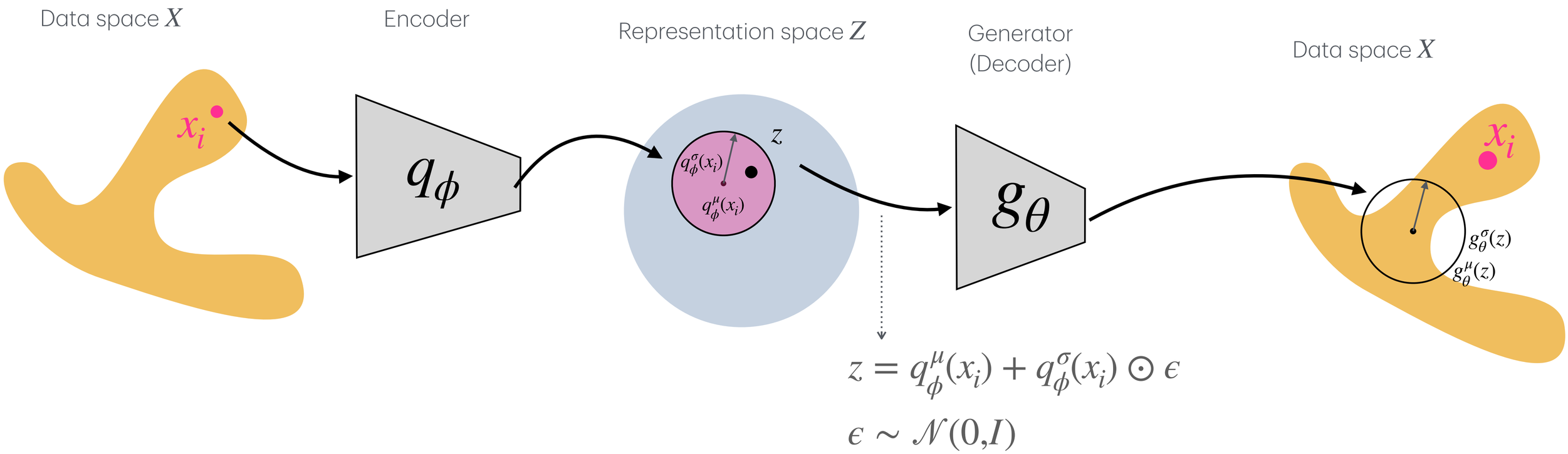
$$\log p_\theta(x_i) = \log \int p_\theta(x_i, z)\, dz = \log \int q_\phi(z \mid x_i) \frac{p_\theta(x_i, z)}{q_\phi(z \mid x_i)} = \log \mathbb{E}_{z \sim q_\phi} \left[\frac{p_\theta(x_i, z)}{q_\phi(z \mid x_i)} \right] \geq \mathbb{E}_{z \sim q_\phi} \left[\log \frac{p_\theta(x_i, z)}{q_\phi(z \mid x_i)} \right]$$

$$= \mathbb{E}_{z \sim q_\phi} \left[\log \frac{p_\theta(x_i \mid z) p(z)}{q_\phi(z \mid x_i)} \right] = \mathbb{E}_{z \sim q_\phi} \left[\log p_\theta(x_i \mid z) \right] + \mathbb{E}_{z \sim q_\phi} \left[\log \frac{p(z)}{q_\phi(z \mid x_i)} \right] = \mathbb{E}_{z \sim q_\phi} [\log p_\theta(x_i \mid z)] - D_{\mathrm{KL}} \left(q_\phi(z \mid x_i) \,\|\, p(z) \right)$$

To maximize $\log p_\theta(x_i)$, we just need to maximize $\rightarrow \mathscr{L}_{\mathrm{ELBO}}(x_i) = \mathbb{E}_{z \sim q_\phi} \left[\log p_\theta(x_i \mid z) \right] - D_{\mathrm{KL}} \left(q_\phi(z \mid x_i) \| p(z) \right)$

Step 4: Importance Sampling instead of naive Montecarlo Sampling

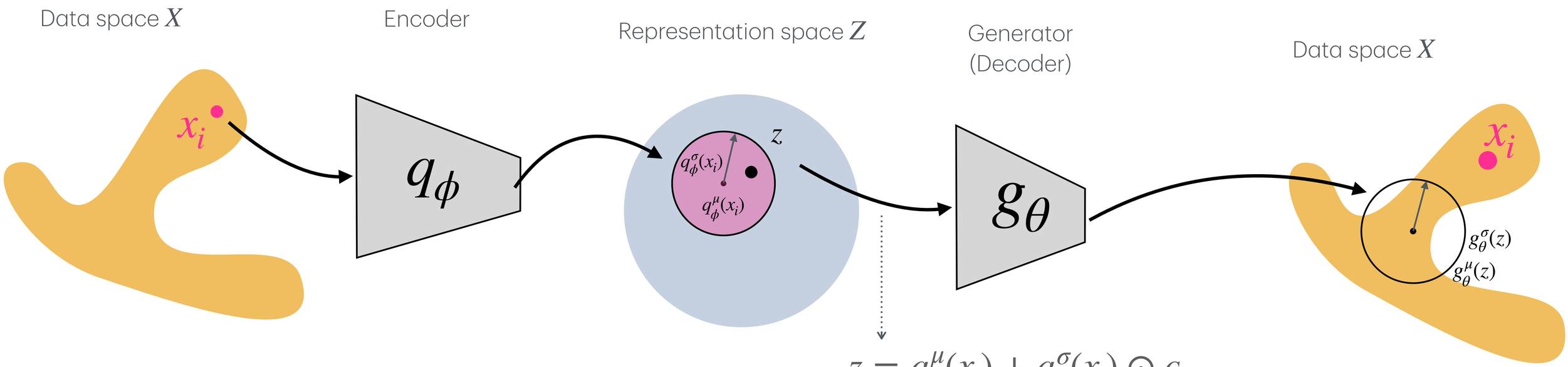
Putting it all together, this shows one forward path of the model:



Maximizing $\mathscr{L}_{\text{ELBO}}(x_i) = \mathbb{E}_{z \sim q_\phi}\left[\log p_\theta(x_i \mid z)\right] - D_{\text{KL}}\left(q_\phi(z \mid x_i) \| p(z)\right)$

Step 4: Importance Sampling instead of naive Montecarlo Sampling

Putting it all together, this shows one forward path of the model:



$$z = q_\phi^\mu(x_i) + q_\phi^\sigma(x_i) \odot \epsilon$$

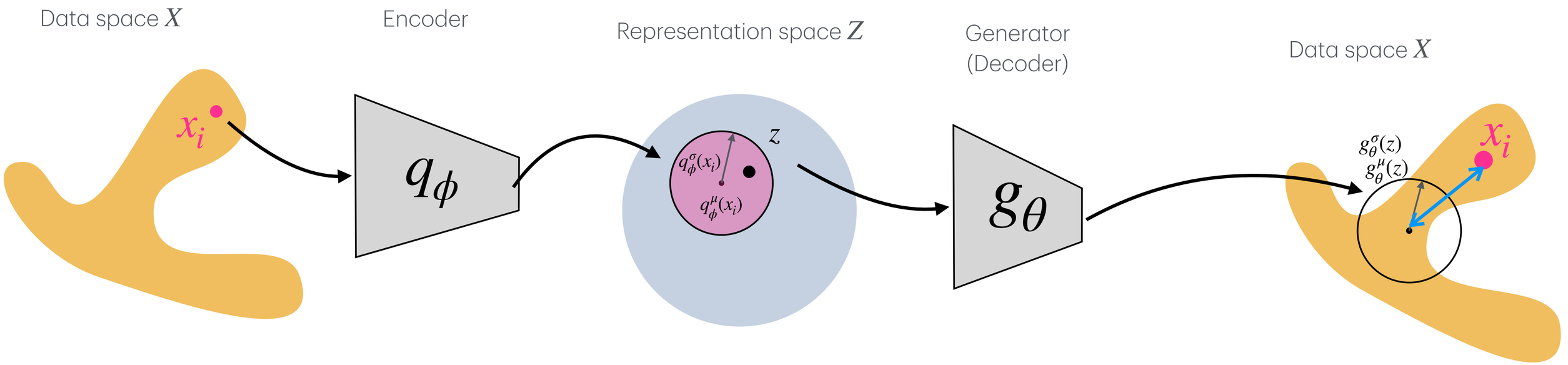
$$\epsilon \sim \mathcal{N}(0, I)$$

for example: $\epsilon = 0.123$

Maximizing $\mathscr{L}_{\text{ELBO}}(x_i) = \mathbb{E}_{z \sim q_\phi}\left[\log p_\theta(x_i \mid z)\right] - D_{\text{KL}}\left(q_\phi(z \mid x_i) \| p(z)\right)$

Step 4: Importance Sampling instead of naive Montecarlo Sampling

Putting it all together, this shows one forward path of the model. This is called Variational Auto Encoder (**VAE**)



Make q_ϕ closer to the $p(z)$
Which is usually a gaussian

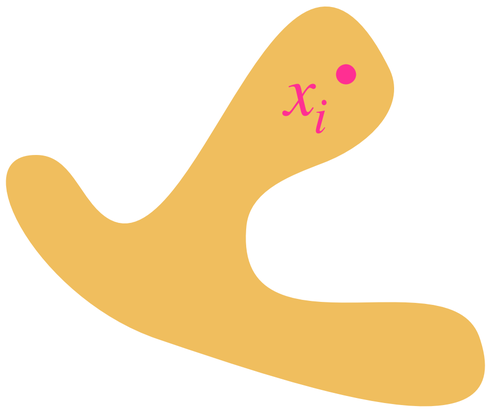
Just a reconstruction loss, minimizing the distance of prediction to x_i

Maximizing $\mathscr{L}_{\text{ELBO}}(x_i) = \mathbb{E}_{z\sim q_\phi}\left[\log p_\theta(x_i \mid z)\right] - D_{\text{KL}}\left(q_\phi(z \mid x_i) \| p(z)\right)$

Step 4: Importance Sampling instead of naive Montecarlo Sampling

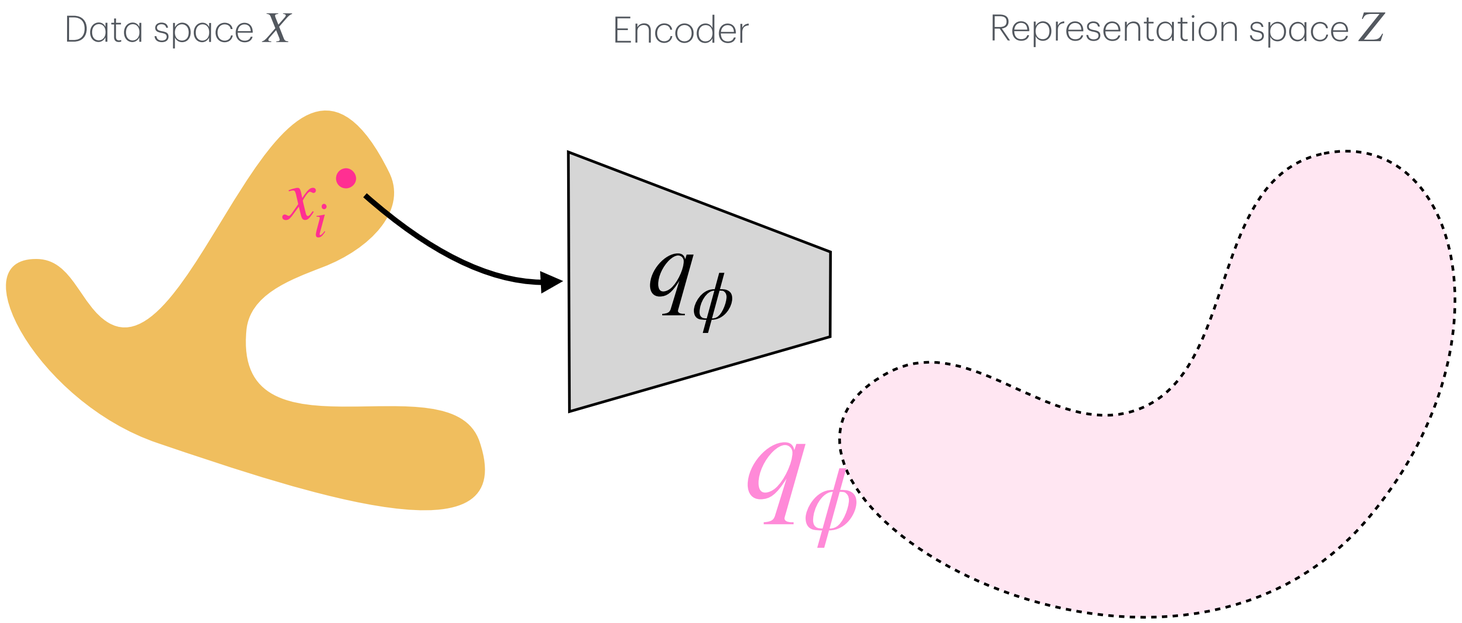
Hypothetical Training: forward path

Data space X



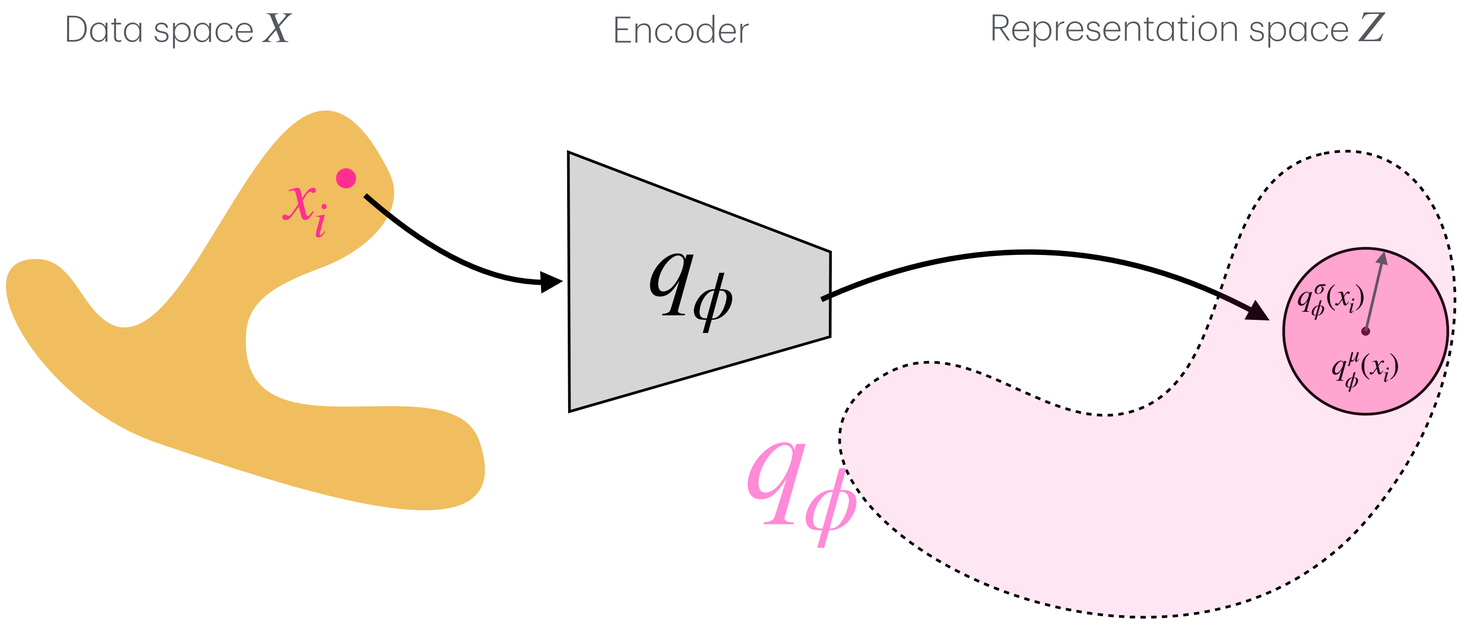
Step 4: Importance Sampling instead of naive Montecarlo Sampling

Hypothetical Training: forward path



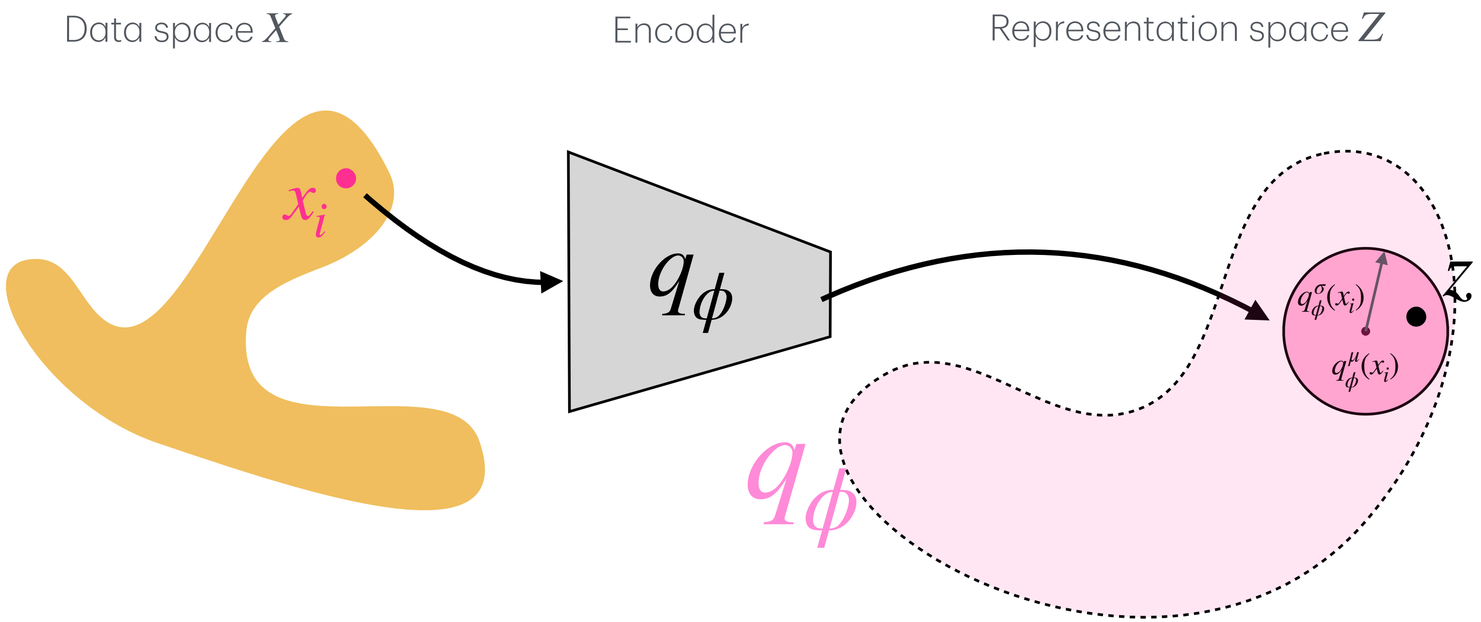
Step 4: Importance Sampling instead of naive Montecarlo Sampling

Hypothetical Training: forward path



Step 4: Importance Sampling instead of naive Montecarlo Sampling

Hypothetical Training: forward path

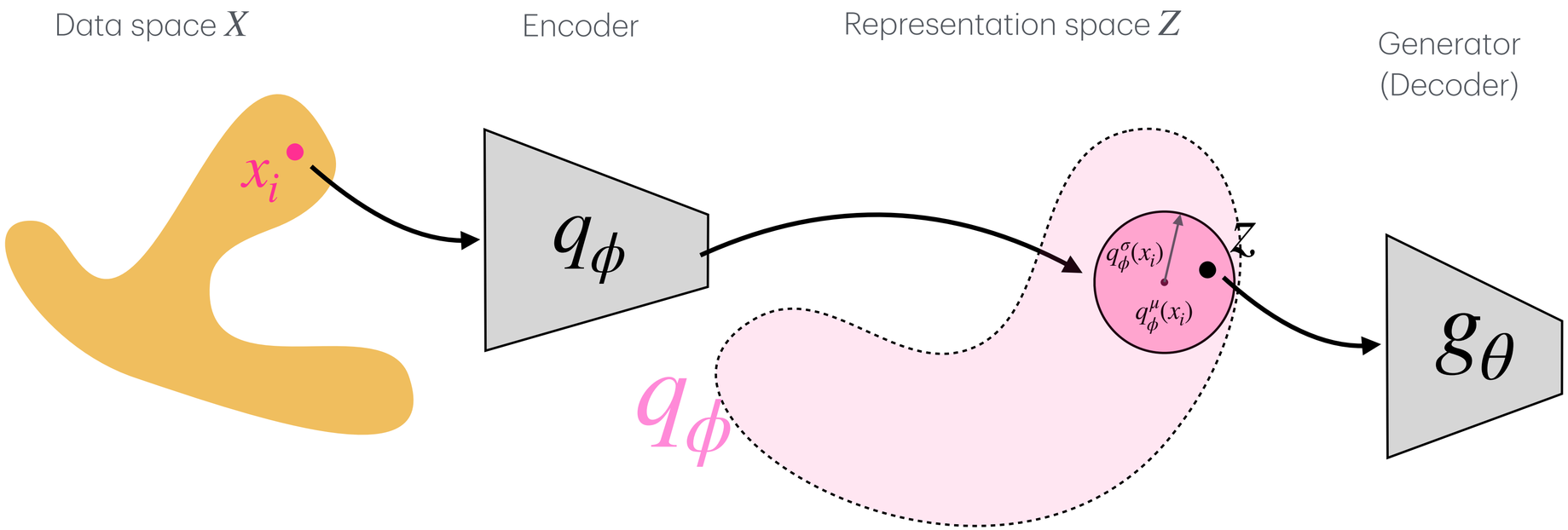


$$z = q_\phi^\mu(x_i) + q_\phi^\sigma(x_i) \odot \epsilon$$

$$\epsilon \sim \mathcal{N}(0, I)$$

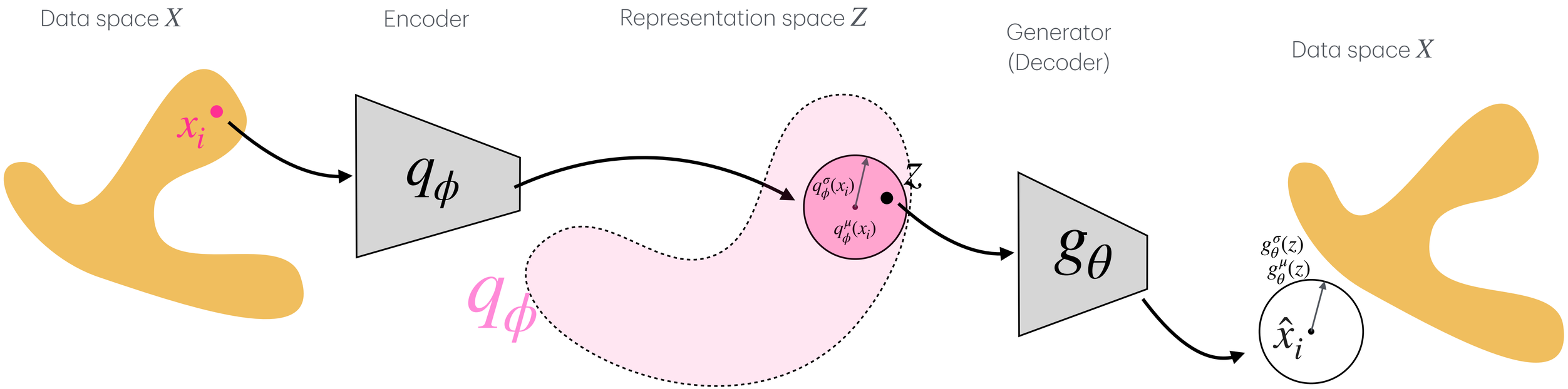
Step 4: Importance Sampling instead of naive Montecarlo Sampling

Hypothetical Training: forward path



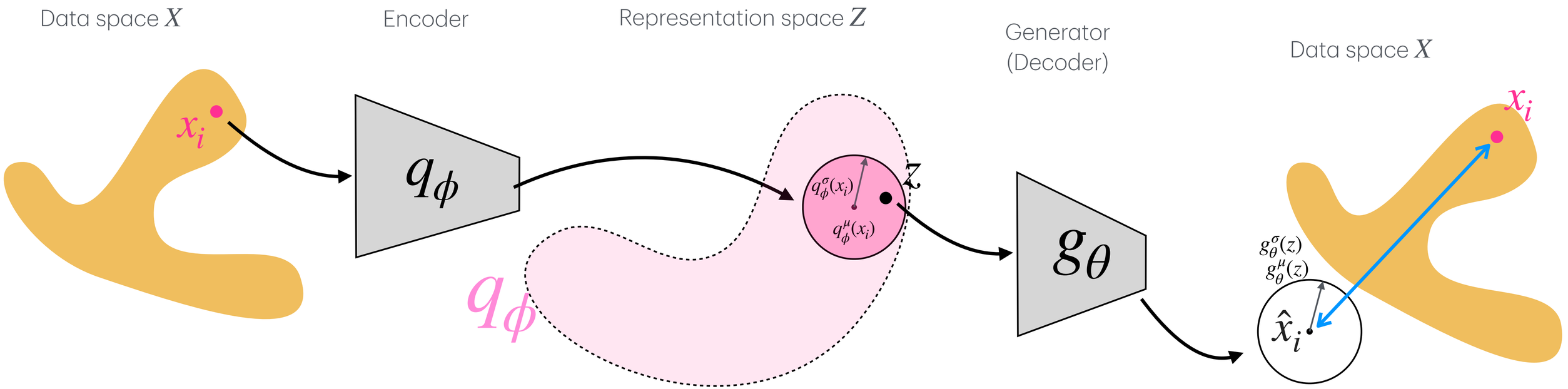
Step 4: Importance Sampling instead of naive Montecarlo Sampling

Hypothetical Training: forward path



Step 4: Importance Sampling instead of naive Montecarlo Sampling

Hypothetical Training: back propagation

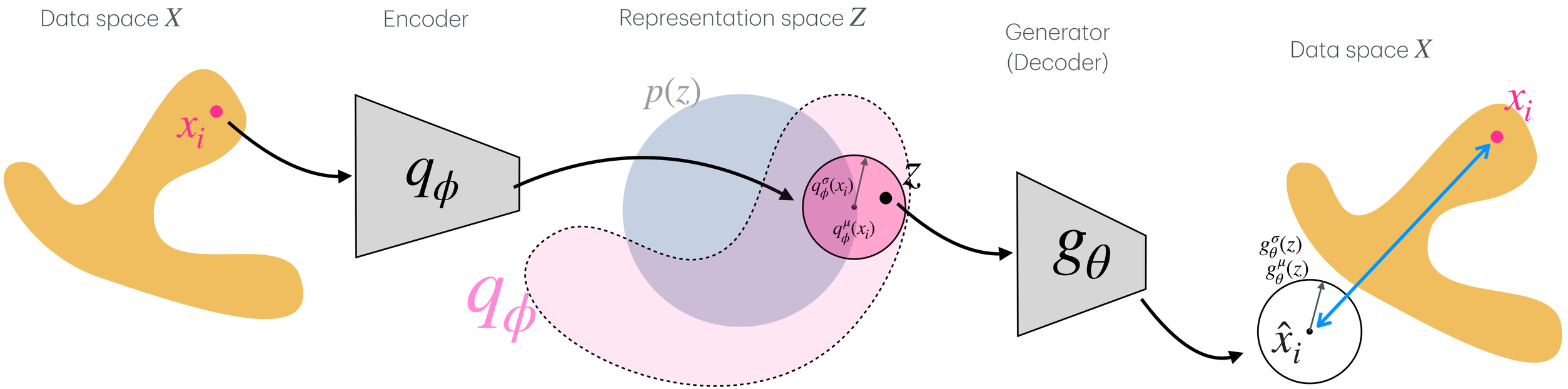


Just a reconstruction loss, minimizing the distance of prediction to x_i

Maximizing $\mathscr{L}_{\text{ELBO}}(x_i) = \mathbb{E}_{z\sim q_\phi}\left[\log p_\theta(x_i \mid z)\right] - D_{\text{KL}}\left(q_\phi(z \mid x_i) \| p(z)\right)$

Step 4: Importance Sampling instead of naive Montecarlo Sampling

Hypothetical Training: back propagation



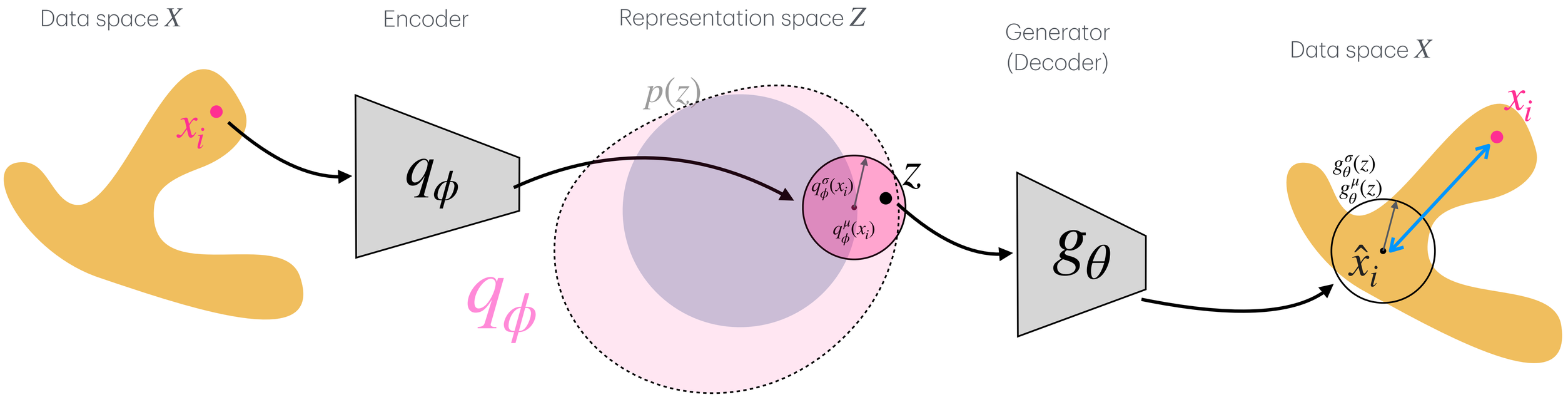
Make q_ϕ closer to the $p(z)$
Which is usually a gaussian

Just a reconstruction loss, minimizing the distance of prediction to x_i

$$\text{Maximizing } \mathscr{L}_{\text{ELBO}}(x_i) = \mathbb{E}_{z\sim q_\phi}\left[\log p_\theta(x_i \mid z)\right] - D_{\text{KL}}\left(q_\phi(z \mid x_i) \| p(z)\right)$$

Step 4: Importance Sampling instead of naive Montecarlo Sampling

Hypothetical Training: Optimization step



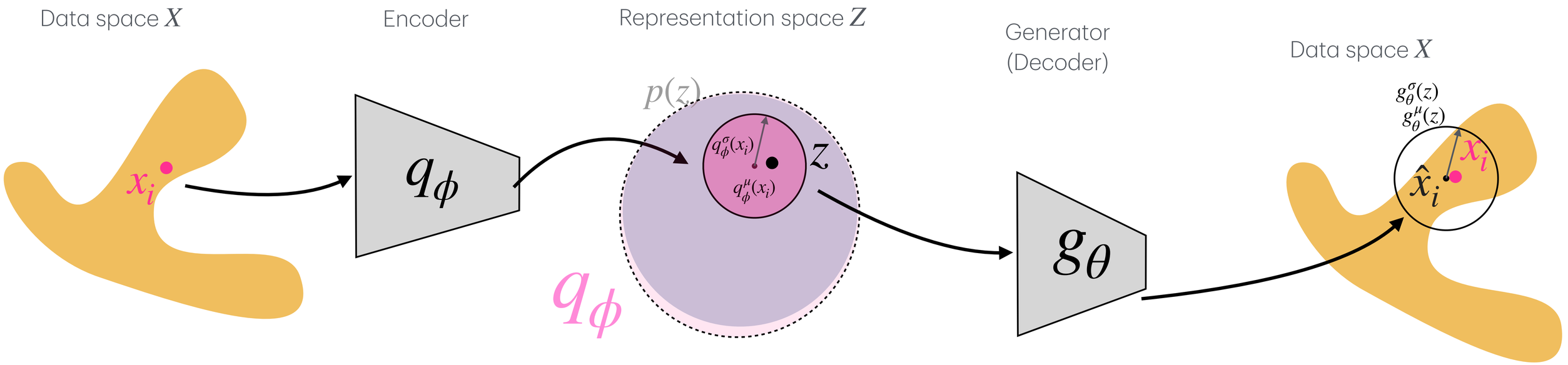
Make q_ϕ closer to the $p(z)$
Which is usually a gaussian

Just a reconstruction loss, minimizing
the distance of prediction to x_i

Maximizing $\mathscr{L}_{\text{ELBO}}(x_i) = \mathbb{E}_{z\sim q_\phi}\left[\log p_\theta(x_i \mid z)\right] - D_{\text{KL}}\left(q_\phi(z \mid x_i) \| p(z)\right)$

Step 4: Importance Sampling instead of naive Montecarlo Sampling

Hypothetical Training: After many Forward → backpropagation → optimization



Make q_ϕ closer to the $p(z)$
Which is usually a gaussian

Just a reconstruction loss, minimizing the distance of prediction to x_i

$$\text{Maximizing } \mathscr{L}_{\text{ELBO}}(x_i) = \mathbb{E}_{z\sim q_\phi}\left[\log p_\theta(x_i \mid z)\right] - D_{\text{KL}}\left(q_\phi(z \mid x_i)\|p(z)\right)$$

Why does this work better than naive Monte Carlo?

1. The gradient is now a distance, not a random Monte Carlo needed lucky overlaps.

Naive Monte Carlo:

Sample naively from $z^{(m)} \sim p(z)$ and only useful samples that contribute to gradient had large $p_\theta(x_i \mid z^{(m)})$

Variational Inference / ELBO

$x_i \rightarrow q_\phi(z \mid x_i) \rightarrow z \rightarrow g_\theta(z) \rightarrow x_i'$

the reconstruction term gives a direct signal and even if the reconstruction is bad, the model still knows which direction to move.

2. It stays generative:

The KL term keeps the encoder distribution compatible with the prior:

$$D_{\mathrm{KL}}\left(q_\phi(z \mid x_i) \| p(z)\right)$$

So after training, we can still generate novel and meaningful data from the prior and generate:

$$z_{new} \sim p(z)$$

$$x_{new} \sim P_\theta(x \mid z_{new})$$

Recipe Card: **VAEs**

Type of Learning: **Unsupervised**

Data set: **Any data**

Data

1- Load libraries: Basics + PyTorch

2-Load your Dataset: $D = \{x_i\}_{i=1}^{N}$

3-Now set up the DataLoader for training/validation/testing:

-Standard

NN model

4-Building the VAE neural network:

-The VAE will have 4 main parts:

A) [Encoder Network] or q_ϕ:

Input, x_i→[NN layers of choice based on data]→⋯→[hidden representation →

(Example NNs: MLP for vectors, CNN for images, GNN for graphs, DeepSet for sets)

B) [Latent Parameter Layer]:

$$\rightarrow \left(\mu = q_\phi^\mu(x_i), \sigma = q_\phi^\sigma(x_i)\right) \rightarrow$$

C) [Reparameterization Layer]:

$$z_i = \mu_i + \sigma_i \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0,I)$$

ϵ is selected from a distribution $\mathcal{N}$, is a real number and not learnable.

D) [MLP Decoder Network]:

z→[NN layers of choice based on data]→⋯→ out, $\hat{x}_i$

-The whole VAE network will be:

x_i→**[Encoder Network] →[Latent Parameter Layer]→[Reparameterization Layer]→[MLP Decoder Network]→**$\hat{x}_i$

-Note: σ and μ will have the same dimensions as Z. Because each element of Z will be sampled based on the mean and variance that are encoded for that element in σ and μ.

Training

5-Define a proper loss function

-the loss is the sum of two parts:

(1) Reconstruction loss

(2) KL divergence of the latent space to the prior

$$\mathcal{L} = \text{Reconstruction}(\hat{x}_i, x_i) + \beta D_{KL}\left(q(z \mid x_i) \,\|\, p(z)\right)$$

-Reconstruction loss depends on the data structure

-Usually the prior is normal, but can also vary:

$$p(z) = \mathcal{N}(0,I)$$

-Add regularization (to network) to prevent overfitting

-The loss will be the sum of all relevant losses

6-Define the train and test functions:

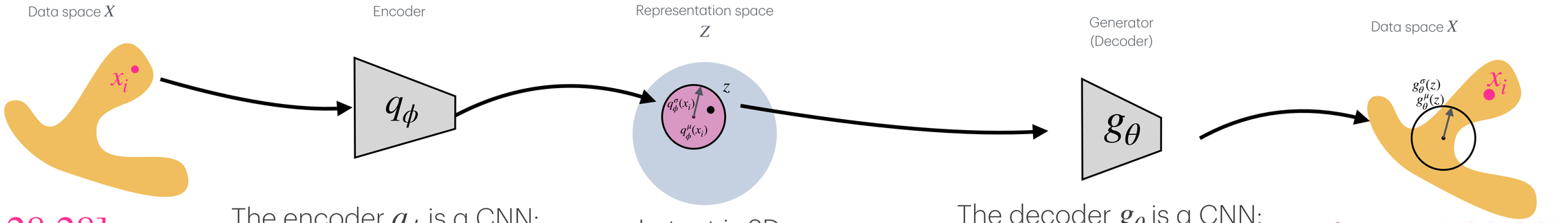
-train/Validate/test functions are similar to standard

-β Should be tuned (usually between 0-1)

7-Define training and run the training:

-same as standard

Example: training of VAE



$x_i : [B,1,28,28]$

The encoder q_ϕ is a CNN:

$[B,1,28,28] \rightarrow [B,32,14,14]$

$[B,32,14,14] \rightarrow [B,64,7,7]$

$[B,64,7,7] \rightarrow [B,3136]$

Latent is 2D:

$Z : [B,2]$

The decoder g_θ is a CNN:

$[B,2] \rightarrow [B,400]$

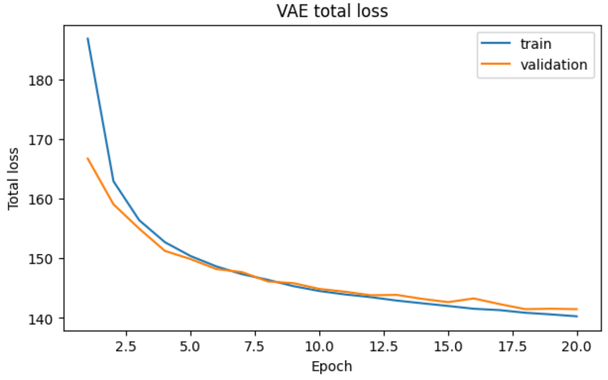
$[B,400] \rightarrow [B,3136]$

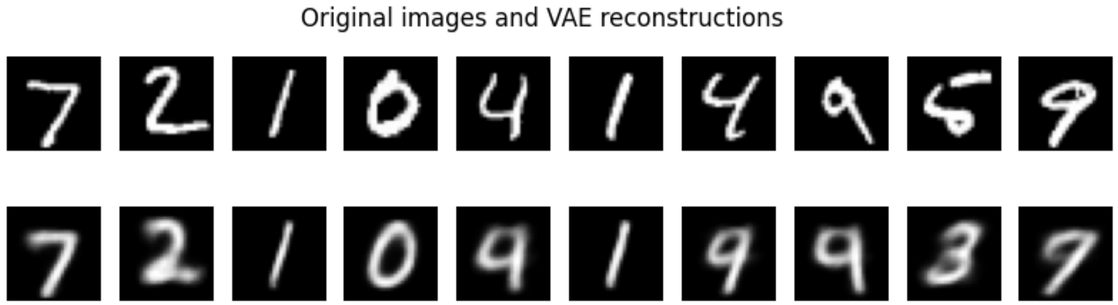
$[B,3136] \rightarrow [B,64,7,7]$

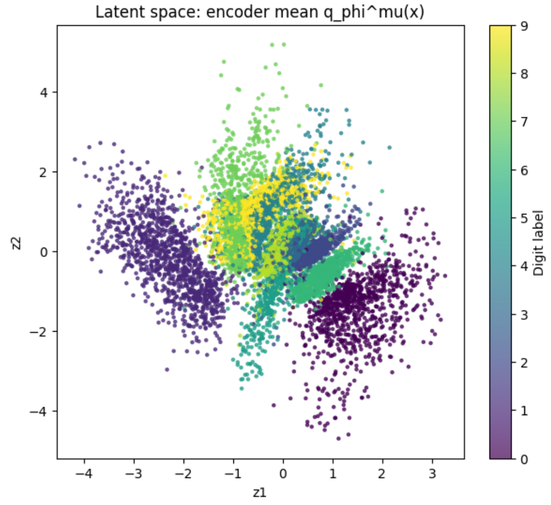
$[B,64,7,7] \rightarrow [B,32,14,14]$

$[B,32,14,14] \rightarrow [B,1,28,28]$

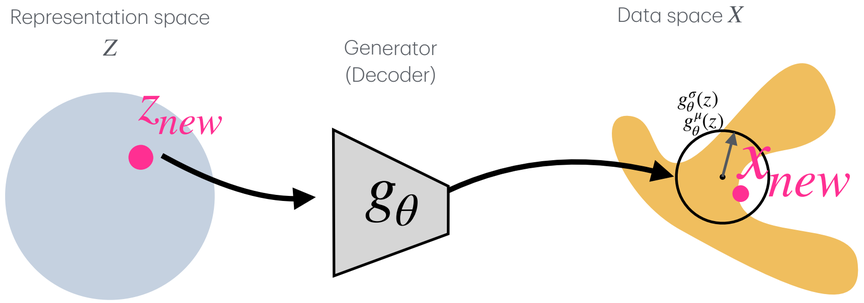
$\hat{x}_i : [B,1,28,28]$







Example: Exploring generation with VAE



New samples:

We traverse the 2D latent space by scanning a grid of points:

$$z_{new} = (z_1, z_2), \qquad z_1, z_2 \in [-3,3],$$

Decoding each point as:

$$\hat{x_{new}} = g_\theta(z_{new}),$$